Deque on a Doubly Circular Linked List

Line-by-line viva guide: deque.h + deque.cpp + main.cpp

Every line of the program explained: what it does and why it is there.

With pictures of the nodes for every check, flowcharts of the checks, a full dry run and a viva question bank.

The program: a **double-ended queue** built on a **fixed doubly circular list of 6 nodes** (maxsize = 6). Every node has **prev** and **next**, so the deque can move one step forward or one step back. Up to 100 deques (MAXS = 100).

Line numbers match the three files sent on 28 October 2026. If you add or remove lines, the numbers move.

Contents

Click a line to jump to that part. In Word you can also open View > Navigation Pane to see every heading.

1. How to use this guide

2. The big picture

- 2.1 What a deque is
- 2.2 How a deque is stored: a doubly circular list of 6 nodes
- 2.3 Singly ring or doubly ring?
- 2.4 Empty and full
- 2.5 Many deques at once
- 2.6 How the program runs
- 2.7 The rules this code follows

3. deque.h line by line

4. main.cpp line by line

5. The helper function: createList

6. The four operations

- 6.1 dpushFront: insert at the front
- 6.2 dpushRear: insert at the rear
- 6.3 dpopFront: delete from the front
- 6.4 dpopRear: delete from the rear

7. Display, display backward, count, search and reverse

- 7.1 ddisplay
- 7.2 ddisplayBack (rear to front)
- 7.3 dcount and dsearch
- 7.4 dreverse

8. The question functions (Q1 to Q10)

- 8.1 Q1: splitDeque (split into two deques)
- 8.2 Q2: devenOdd (even and odd deques)
- 8.3 Q3: dnextGreater (next greater element)
- 8.4 Q5: dsortList (bubble sort)
- 8.5 Q6: disPalindrome (palindrome check)
- 8.6 Q7: copyDeque (copy)
- 8.7 Q8: disBalanced (balanced brackets)
- 8.8 Q9: dremoveDuplicates (remove duplicates)
- 8.9 Q10: dintersection (values in both deques)

9. The menu: dequeMenu and menu

- 9.1 The key idea: references to the selected deque
- 9.2 dequeMenu line by line
- 9.3 menu

10. A complete dry run

11. Viva question bank

- 11.1 Concepts
- 11.2 Pointers and checks
- 11.3 Questions about specific lines
- 11.4 C++ questions
- 11.5 What-if questions

12. One-page revision sheet

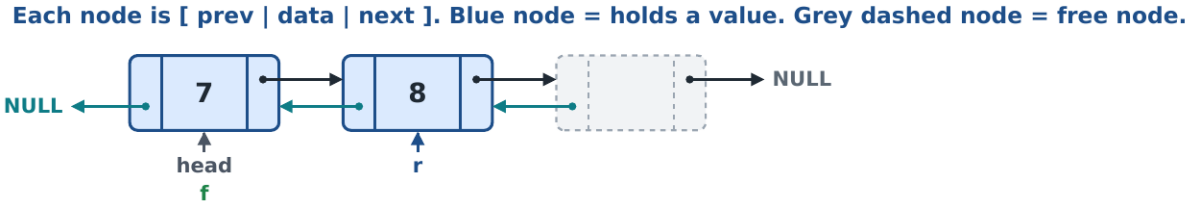
1. How to use this guide

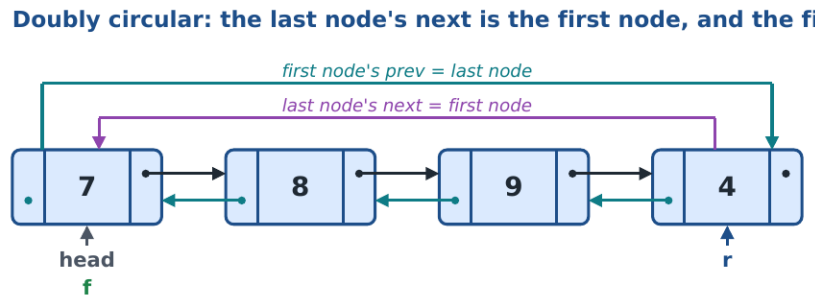
Read part 2 first: it explains the idea of the whole program in a few pages. Then each function has the same layout:

- **What it does**: one or two sentences.
- **Parameters**: every parameter, whether it is passed by value or by reference, and why the function needs it.
- **Pictures**: the nodes before and after, and for the four main operations a flowchart of the checks.
- **Line by line**: a table with the line number, the code and what that line does.
- **Viva questions** for that function, with short answers.

Part 10 is a complete dry run, part 11 is a viva question bank and part 12 is a one-page revision sheet.

How to read the pictures





Black arrows (top) are next links, going right. Teal arrows (bottom) are prev links, going left. The purple arc is the last node's next and the teal arc is the first node's prev. Pointers are written under the node they point at: head (grey), f = front (green), r = rear (blue). In the flowcharts, a yellow box is a check, green means the normal action, red means an Overflow or Underflow message.

Two words used everywhere

Word	Meaning in this program
by value	The function gets a **copy** of the pointer. It can change the data inside the nodes, but it cannot move the caller's pointer. Written `node *f`.
by reference	The function gets the **caller's own** pointer. When the function moves it (for example `f = head;`), the caller sees the change. Written `node *&f`.

2. The big picture

2.1 What a deque is

A **deque** (double-ended queue, said "deck") is a list where you can **insert and delete at both ends**: the front and the rear.

Structure	Insert at	Delete from
Stack	top	top
Queue	rear	front
Circular queue	rear (wraps around)	front (wraps around)
Deque	**front and rear**	**front and rear**

So a deque can act as a stack (insert and delete at the same end) or as a queue (insert at the rear, delete at the front). It has four main operations:

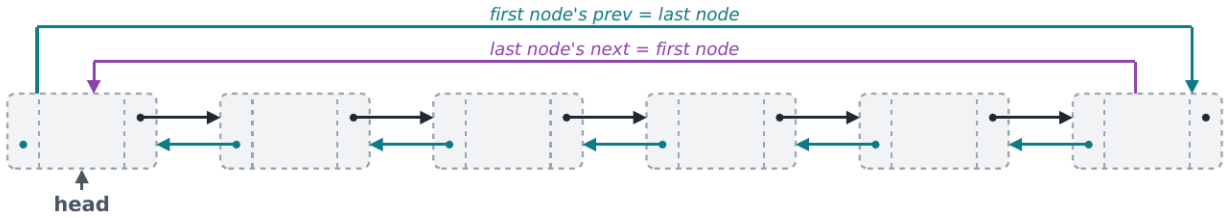
Operation	Function	Pointer that moves	Direction	Link used
Insert at front	`dpushFront`	f	**backward** (one node back)	`prev`
Insert at rear	`dpushRear`	r	forward (one node on)	`next`
Delete from front	`dpopFront`	f	forward	`next`
Delete from rear	`dpopRear`	r	**backward**	`prev`

> **The one idea to remember**
>
> A deque must move its pointers **both ways**. Moving forward uses `next`. Moving backward uses `prev`. Because every node of a **doubly** linked list knows the node before it, the backward moves (insert at front, delete from rear) are **one step**: `f = f->prev` or `r = r->prev`. No search and no helper function are needed. That is why a doubly linked list suits a deque.

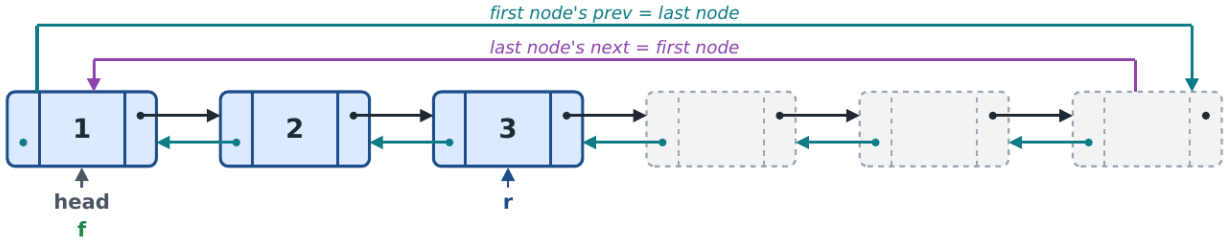
2.2 How a deque is stored: a doubly circular list of 6 nodes

`createList(head)` builds **6 nodes** once, at the start. Each node is linked to the next one **and** to the previous one. Then the last node's `next` is joined to the first node, and the first node's `prev` is joined to the last node. That makes a **doubly circular list** (a ring in both directions). After that, no node is ever created or deleted. Insert and delete only **move the pointers f and r** and write values into the nodes.

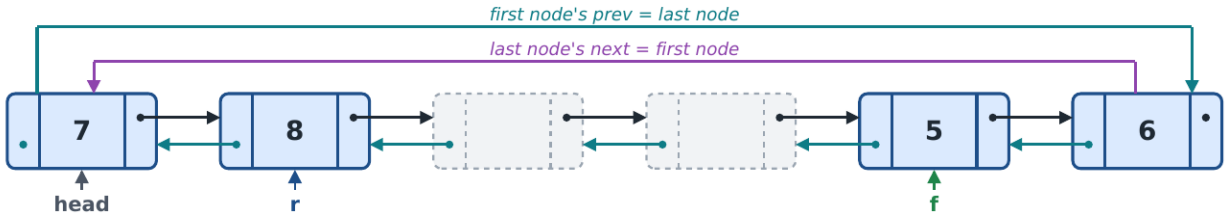
Empty deque: f and r are NULL. All 6 nodes exist but are free.



3 values: the used nodes go from f forward to r. The rest are free.



Wrapped: f is on node 5 and r is on node 2. Front to rear reads 5 6 7 8.



The used part always goes from f forward to r, and it may pass through the last node back to the first node. That is the wrap. Going backward from r with prev reaches f the same way.

2.3 Singly ring or doubly ring?

Your earlier deque used a **singly** circular list (only `next`). This one uses a **doubly** circular list. The checks are the same; only the backward moves change.

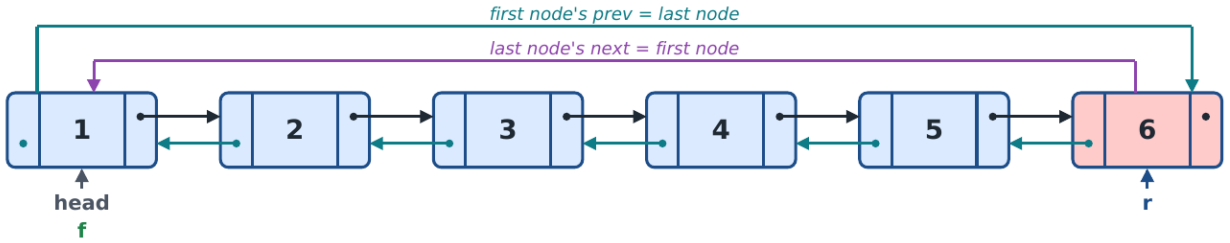
	Singly circular (old)	Doubly circular (this program)
Links in a node	`next` only	`next` and `prev`
Move f or r back one node	search from head with `prevNode` (walks up to n nodes)	`f = f->prev` (one step)
Back from the first node	a loop finds the last node	`head->prev` is the last node
Time of insert front / delete rear	O(n)	**O(1)**
Time of insert rear / delete front	O(1)	O(1)
Helper functions	createList and prevNode	createList only
Memory per node	data + 1 pointer	data + **2** pointers
Display backward	hard	easy: follow `prev` from r

2.4 Empty and full

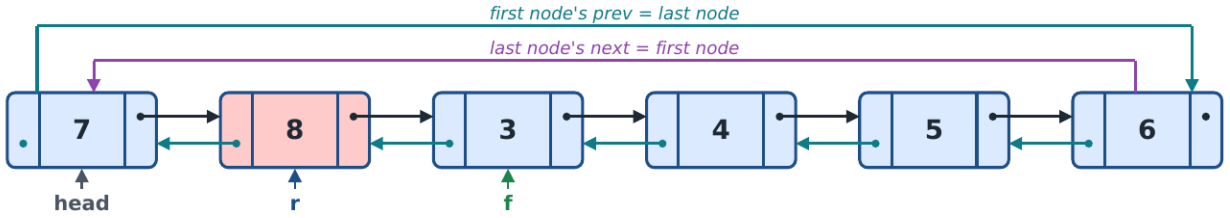
State	Condition in the code	Picture
Empty	`f == NULL` (and `r == NULL`)	no value at all
One value	`f == r`	front and rear on the same node

State	Condition in the code	Picture
Full, not wrapped (simple)	`f == head && r->next == head`	f on the first node, r on the last node
Full, wrapped (circular)	`r->next == f`	front is one node after rear: front = rear + 1

Full simple deque: f == head and r->next == head. Message: Deque Overflow



Full circular deque: r->next == f. Message: Circular Deque Overflow



Both pictures hold 6 values. The program checks the simple case first so the message tells you which kind of full it was.

2.5 Many deques at once

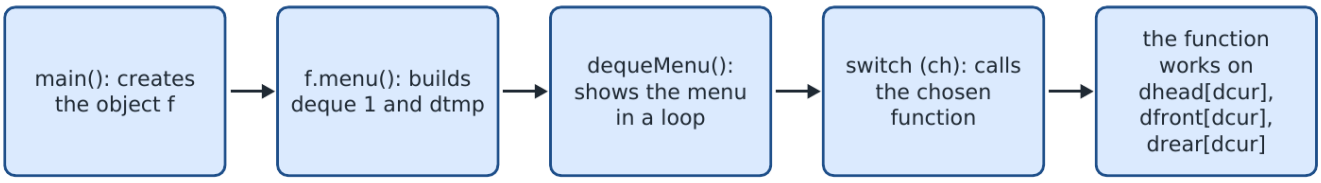
The program can hold up to **100 deques**. Deque number i is described by three array slots: `dhead[i]`, `dfront[i]` and `drear[i]`. The user sees numbers starting at 1, the arrays start at 0.

User number	Array index	dhead[i]	dfront[i]	drear[i]
Deque 1	0	first node of list 1	front of deque 1	rear of deque 1
Deque 2	1	first node of list 2	front of deque 2	rear of deque 2
...	...	...	...	...
Deque 100	99	first node of list 100	front	rear

- `dnum` = how many deques exist now. `dcur` = index of the deque you are working on (the selected one).
- `dtmp` = one extra list used only inside palindrome, brackets and remove-duplicates, so your own deques are not disturbed.
- Questions that make new deques (split, even/odd, copy, intersection) put them at index `dnum` and then increase `dnum`.

2.6 How the program runs

From main() to a function



2.7 The rules this code follows

- No `!` (not) operator and no `bool`: every test is a plain comparison, and yes/no answers are returned as 1 or 0.
- Only one helper function: `createList`. The old `prevNode` is gone because every node has `prev`.
- Every check is written separately and numbered (Check 1, Check 2 ...) with its own message, so you can tell which one ran.
- All inserts go through the base functions `dpushFront` and `dpushRear`, so the overflow checks always apply.
- Results of questions are new deques you can select and keep using.

3. deque.h line by line

The header file **declares** everything: the node, the struct `func` with all the arrays and counters, and the list of functions. It contains no function bodies. `deque.cpp` and `main.cpp` both include it.

Line	Code (deque.h)	What this line does, and why
1	`#ifndef DEQUE_H`	Include guard, part 1: if the name DEQUE_H is **not defined yet**, read the file. Stops the header being included twice, which would give 'redefinition' errors.
2	`#define DEQUE_H`	Include guard, part 2: define DEQUE_H now, so a second include of this file is skipped.
4	`#include <iostream>`	For `cout` (printing) and `cin` (reading input).
5	`#include <string>`	For the `string` type, used by the palindrome and bracket questions.
6	`using namespace std;`	Lets us write `cout`, `cin`, `string`, `endl` without `std::` in front.
8	`const int MAXS = 100;  // most deques that can exist`	The most deques that can exist. It is the size of the arrays below. `const` means it can never change while the program runs.
10	`struct node`	Start of the node type. Every deque is made of these.
12	`  int data;`	The value stored in the node.
13	`  node *next;  // link to the node after`	Pointer to the **next** node (forward). The last node's `next` points back to the first node.
14	`  node *prev;  // link to the node before`	Pointer to the **previous** node (backward). This is what makes the list **doubly** linked. The first node's `prev` points to the last node.
17	`struct func`	One struct that holds **all the deques and all the functions**. main() makes one object of it.
19	`  int maxsize = 6;  // nodes in every deque`	Number of nodes in **every** deque. Change it here and every deque gets that size. It is also the overflow limit.
22	`  node *dhead[MAXS] = {NULL};`	Array of 100 pointers: `dhead[i]` is the **first node** of deque i's list. All start as NULL.
23	`  node *dfront[MAXS] = {NULL};`	Array of 100 pointers: `dfront[i]` is the **front** of deque i. NULL means that deque is empty.
24	`  node *drear[MAXS] = {NULL};`	Array of 100 pointers: `drear[i]` is the **rear** of deque i.
25	`  int dnum = 0;  // how many deques exist`	How many deques exist right now. menu() sets it to 1 when it builds deque 1.
26	`  int dcur = 0;  // index of the selected deque`	Index of the **selected** deque. The menu always works on deque `dcur`.
29	`  node *dtmp = NULL;`	Head of a temporary list used inside palindrome, brackets and remove-duplicates.

Line	Code (deque.h)	What this line does, and why
31	`void createList(node *&head);`	The only helper: builds a doubly circular list of maxsize nodes. `head` is by reference because the function creates the first node and hands it back.
34	`void dpushFront(node *head, node *&f, node *&r, int x);`	Insert x at the front. f and r are by reference because the function moves them.
35	`void dpushRear(node *head, node *&f, node *&r, int x);`	Insert x at the rear.
36	`int dpopFront(node *head, node *&f, node *&r);`	Delete from the front and return the value (-1 if empty).
37	`int dpopRear(node *head, node *&f, node *&r);`	Delete from the rear and return the value (-1 if empty).
38	`void ddisplay(node *f, node *r);`	Print from front to rear. Only reads, so f and r are by value.
39	`void ddisplayBack(node *f, node *r);`	Print from rear to front, following `prev`. Possible because the list is doubly linked.
40	`int dcount(node *f, node *r);`	Return how many values the deque holds.
41	`int dsearch(node *f, node *r, int x);`	Return 1 if x is in the deque, 0 if not.
42	`void dreverse(node *&f, node *&r);`	Reverse the deque by turning the links (f and r by reference because they change).
43	`int splitDeque(node *head, node *f, node *r, node *&h1, node *&f1, node *&r1, node *&h2, node *&f2, node *&r2);`	Q1: copy and cut a deque into two new deques. Outputs (h1, f1, r1) and (h2, f2, r2) are by reference. Returns 1 if done, 0 if it cannot split.
44	`void devenOdd(node *f, node *r, node *eh, node *&evF, node *&evR, node *oh, node *&odF, node *&odR);`	Q2: even values into one new deque (eh, evF, evR), odd values into another (oh, odF, odR).
45	`void dnextGreater(node *f, node *r);`	Q3: print the next greater element of every value (wraps around).
46	`void dsortList(node *f, node *r);`	Q5: bubble sort the values.
47	`int disPalindrome(string s);`	Q6: return 1 if the word is a palindrome.
48	`void copyDeque(node *f, node *r, node *ch, node *&cf, node *&cr);`	Q7: copy deque (f, r) into the deque whose head is ch. The copy's front and rear are by reference.
49	`int disBalanced(string s);`	Q8: return 1 if the brackets are balanced.
50	`void dremoveDuplicates(node *head, node *&f, node *&r);`	Q9: remove repeated values, keeping the first copy.
51	`void dintersection(node *f1, node *r1, node *f2, node *r2, node *rh, node *&rf, node *&rr);`	Q10: values found in both deques go into the result deque (rh, rf, rr).
54	`void dequeMenu();`	The deque menu (the loop you see on the screen).
55	`void menu();`	Start-up: builds deque 1 and dtmp, then opens dequeMenu.
58	`#endif`	End of the include guard that started on line 1.

Q. What makes this list doubly linked?

The extra pointer `node *prev;` in the node (deque.h line 14). Every node points forward with `next` and backward with `prev`.

Q. Why is there an include guard (#ifndef / #define / #endif)?

If deque.h were read twice in one .cpp file, `node` and `func` would be defined twice and the compiler would stop with a redefinition error. The guard makes the second read do nothing.

Q. Why are the deques stored in arrays?

So that one function works for any deque. The menu passes `dhead[dcur]`, `dfront[dcur]` and `drear[dcur]`; changing `dcur` changes which deque is used. No dpush1, dpush2 functions are needed.

Q. What is the difference between maxsize and MAXS?

`maxsize` (6) is the number of **nodes in one deque**. `MAXS` (100) is the number of **deques** that can exist.

Q. Why are function bodies not in the header?

The header only declares. The bodies are in deque.cpp. That is the normal 3-file layout: the header says what exists, the .cpp says how it works, main.cpp uses it.

4. main.cpp line by line

Line	Code (main.cpp)	What this line does, and why
1	`#include "deque.h"`	Brings in the node, the struct func and the function list. Quotes (not < >) because it is our own file in the same folder.
3	`int main()`	Where every C++ program starts.
5	`  func f;`	Makes **one object** f of the struct func. This object holds all the arrays (dhead, dfront, drear), the counters and the functions.
6	`  f.menu();`	Calls menu() on that object: it builds deque 1 and the temporary list, then opens the deque menu loop. The program stays here until the user chooses 0.
7	`  return 0;`	Ends the program and tells the system it finished without error.

> **Compiling the three files**
> Both .cpp files must be compiled together: `g++ -std=c++11 -Wall main.cpp deque.cpp -o deque.exe`. If only deque.cpp is compiled there is no main() and the linker gives `undefined reference to WinMain`. If only main.cpp is compiled, the function bodies are missing.

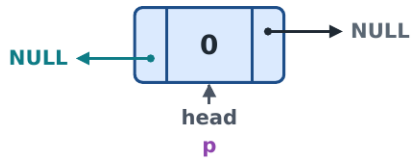
5. The helper function: createList

`deque.cpp` line 1, `#include "deque.h"`, gives this file the declarations from the header. The rest of the file is the function bodies, starting with the only helper.

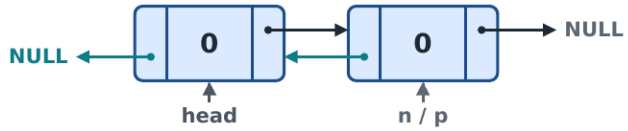
What it does: builds `maxsize` (6) nodes, each with data 0, and links every pair of neighbours **both ways** (`next` forward, `prev` backward). Then it closes the list in both directions: the last node's `next` becomes the first node and the first node's `prev` becomes the last node. Every deque and the temporary list are made by it.

Parameter	Passed how	Why the function needs it
head	by reference (`node *&head`)	The function creates the first node and must give it back to the caller. The caller passes an array slot such as `dhead[0]`, and that slot becomes the first node.

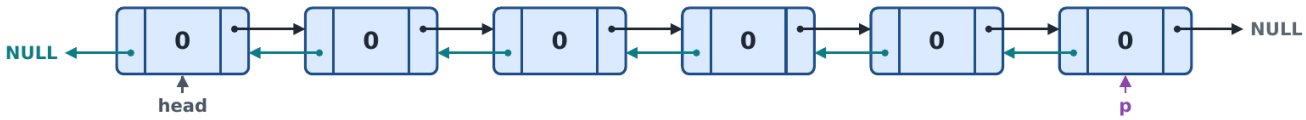
Lines 11-13: the first node is made, head and p point to it.



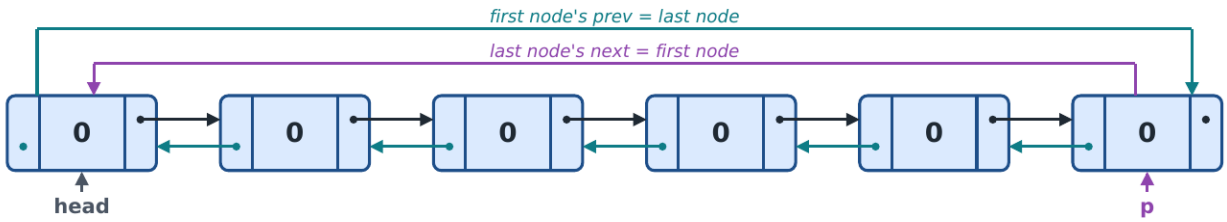
One loop pass (lines 16-20): a new node n, p->next = n, n->prev = p, then p = n.



After the loop: 6 nodes linked both ways. The two ends are still open (the pictures show them as NULL).



Line 22: last node's next = head. Line 23: head's prev = last node. The list is closed both ways.



The value 0 is only a placeholder. A node counts as used only when it is between f and r.

Line	Code (deque.cpp)	What this line does, and why
9	`void func::createList(node *&head)`	Function header. Belongs to struct func (`func::`). Returns nothing.

Line	Code (deque.cpp)	What this line does, and why
11	`head = new node;`	Create the **first node** in memory. The caller's pointer (for example `dhead[0]`) now points to it.
12	`head->data = 0;`	Give it a starting value of 0.
13	`node *p = head;`	p will always point to the **last node made so far**. At the start that is the head.
14	`for (int i = 1; i < maxsize; i++)`	Repeat maxsize - 1 times (5 times), because the first node already exists. Total = 6 nodes.
16	`node *n = new node;`	Create one new node.
17	`n->data = 0;`	Starting value 0.
18	`p->next = n;  // forward link`	**Forward link:** the old last node points to the new node.
19	`n->prev = p;  // backward link`	**Backward link:** the new node points back to the old last node. These two lines together link the pair both ways.
20	`p = n;`	The new node is now the last node.
22	`p->next = head;  // last node -> first node`	After the loop p is the 6th node. Its `next` goes back to the first node: the forward ring is closed.
23	`head->prev = p;  // first node -> last node`	The first node's `prev` goes to the last node: the backward ring is closed. Now `head->prev` gives the last node in one step.

Q. Why is head passed by reference?

The function creates the first node. Without the reference only a local copy of the pointer would change and `dhead[0]` would stay NULL.

Q. Why does the loop start from i = 1?

The first node was already made before the loop, so only maxsize - 1 more nodes are needed.

Q. What do lines 18 and 19 do together?

They link two neighbours in both directions: the old last node points forward to the new node, and the new node points back to the old last node.

Q. What do lines 22 and 23 change?

They close the list in both directions. Moving forward from the last node goes to the first node, and moving backward from the first node goes to the last node. This is what lets f and r wrap both ways.

Q. Is the first node's prev ever left unset?

No. It is set on line 23, after all nodes exist, because only then is the last node known.

Q. Are nodes ever deleted later?

No. All 6 nodes are made once. Insert and delete only move f and r and overwrite data.

Q. Why is there no prevNode function any more?

prevNode searched for the node before p because a singly linked node cannot see backward. Here every node has `prev`, so the node before p is simply `p->prev`.

6. The four operations

These four functions are the heart of the deque. Each one has **numbered checks** that run in order: the first check that is true does its job and returns. All four take the same three pointers: `head` (the first node of the ring), `f` (front) and `r` (rear).

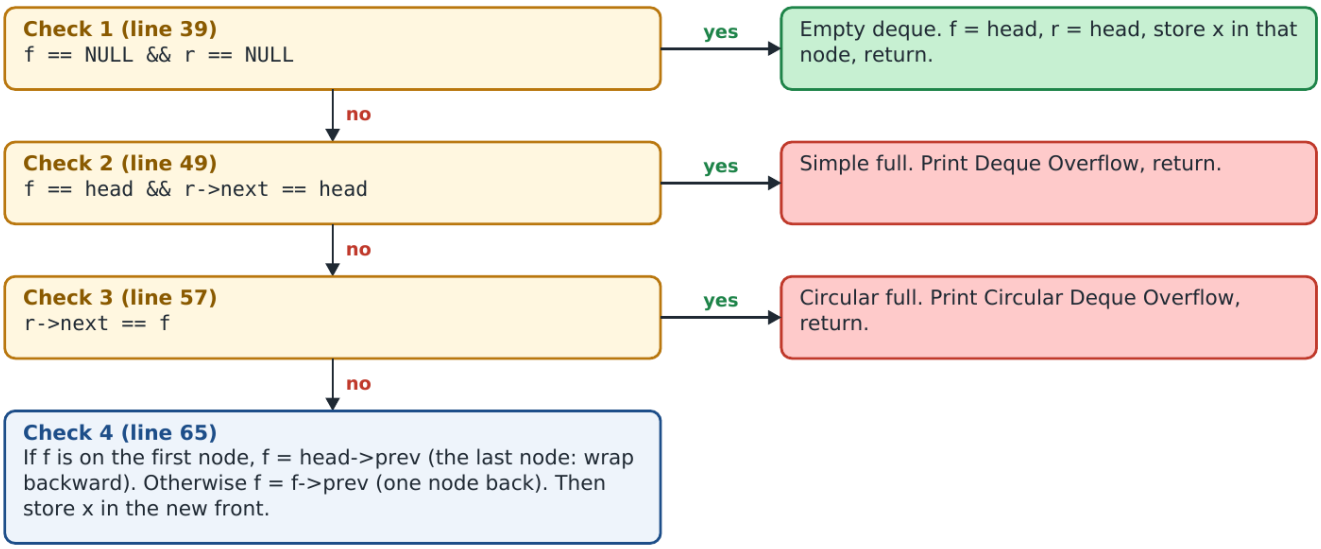
	dpushFront	dpushRear	dpopFront	dpopRear
Check 1	empty: f = r = head, store	empty: f = r = head, store	empty: Deque Underflow	empty: Deque Underflow
Check 2	simple full: Deque Overflow	simple full: Deque Overflow	one value: f = r = NULL	one value: f = r = NULL
Check 3	circular full: Circular Deque Overflow	circular full: Circular Deque Overflow	f on last node: f = head, else forward	r on head: r = head->prev, else r = r->prev
Check 4	f on head: f = head->prev, else f = f->prev	r on last node: r = head, else forward		

6.1 dpushFront: insert at the front

What it does: puts a new value **before** the current front. The front moves **one node back**. If the deque is full it prints an overflow message and changes nothing.

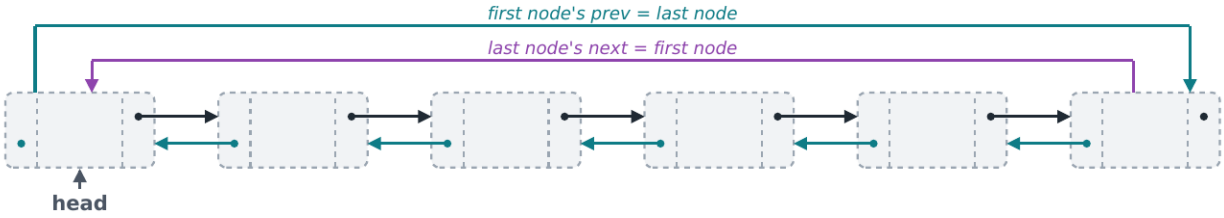
Parameter	Passed how	Why the function needs it
head	by value	The first node of the list. Needed to start an empty deque (f = r = head), to test 'f is on the first node', and to reach the last node in one step (`head->prev`).
f	by reference (`node *&f`)	The front moves back (and is set on the first insert). The caller (`dfront[dcur]`) must see the new front.
r	by reference (`node *&r`)	Set on the first insert (r = head). Also read in the full checks.
x	by value (int)	The value to insert.

dpushFront: the checks in order

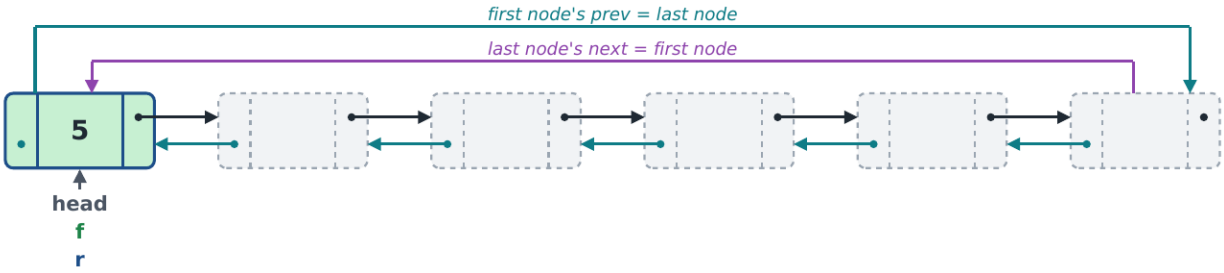


Pictures of every check

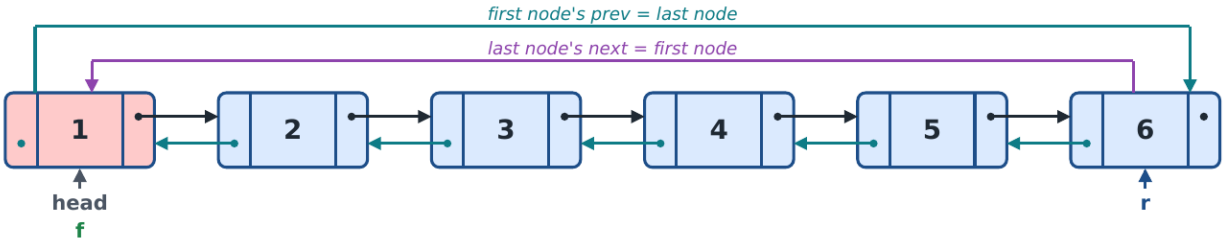
Check 1, before: empty deque (f and r are NULL)



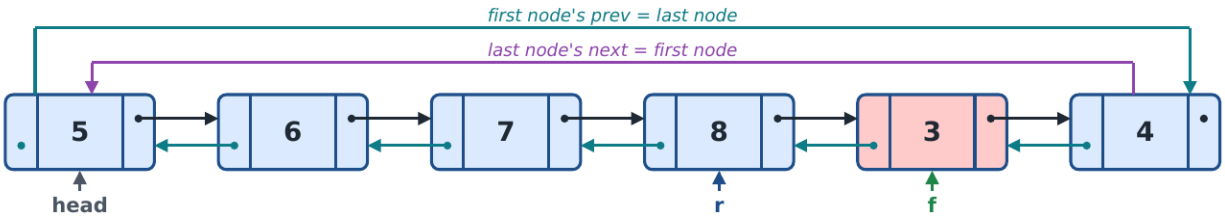
Check 1, after dpushFront(5): f and r both on the first node



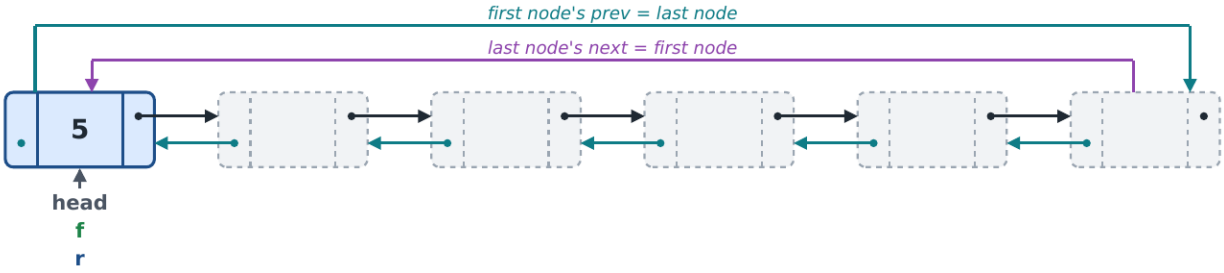
Check 2: f == head and r->next == head, so Deque Overflow. Nothing changes



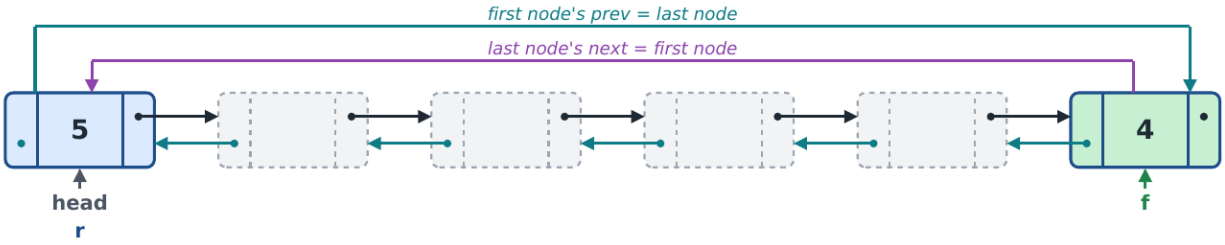
Check 3: r->next == f (rear's next node is the front), so Circular Deque Overflow



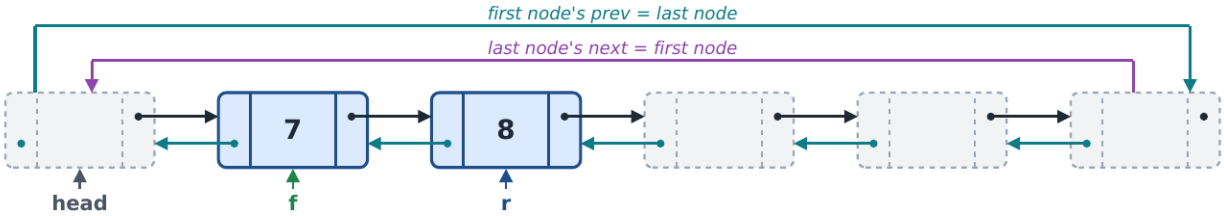
Check 4 (f on the first node), before dpushFront(4)



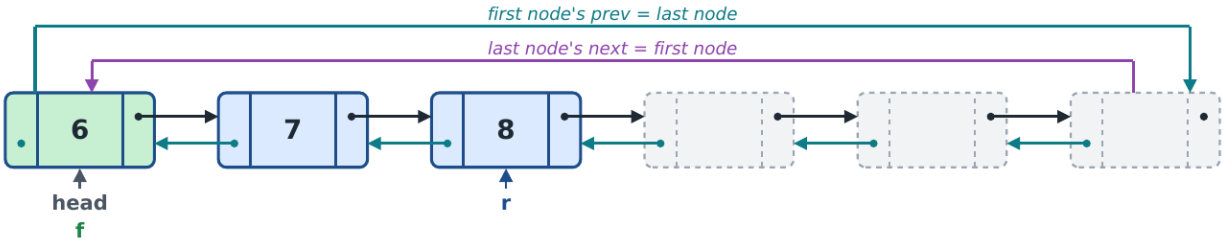
After: line 66 puts f on the last node (head->prev) and 4 is stored. Front to rear reads 4 5



Check 4 (f not on the first node), before dpushFront(6)



After: f = f->prev moves f one node back, then 6 is stored. Reads 6 7 8



Moving the front backward is one step: the node before f is f->prev. In a singly linked list this needed a search from head.

Line by line

Line	Code (deque.cpp)	What this line does, and why
35	`void func::dpushFront(node *head, node *&f, node *&r, int x)`	Function header: insert x at the front. Returns nothing (void).
39	`if (f == NULL && r == NULL)`	**Check 1:** both pointers NULL means the deque is empty.

Line	Code (deque.cpp)	What this line does, and why
41	`f = head;`	The first value always goes in the first node. Front goes there...
42	`r = head;`	...and rear too, because one value is both the front and the rear.
43	`f->data = x;`	Store the value.
44	`return;`	Done; skip the other checks.
49	`if (f == head && r->next == head)`	**Check 2 (simple full):** front on the first node AND the node after rear is the first node (so rear is on the last node). All 6 nodes are used and nothing has wrapped.
51	`cout << "Deque Overflow" << endl;`	Tell the user the deque is full.
52	`return;`	Do not insert.
57	`if (r->next == f)`	**Check 3 (circular full):** the node after rear is the front, meaning front = rear + 1. The used part has wrapped around and filled the whole ring.
59	`cout << "Circular Deque Overflow" << endl;`	Different message, so you can see which check found it full.
60	`return;`	Do not insert.
65	`if (f == head)`	**Check 4, part 1:** is the front on the first node? Then the node before it is the **last** node of the list (wrap backward).
66	`f = head->prev;  // head's prev is the last node`	The first node's `prev` is the last node (set by createList), so the front wraps back to it in one step.
67	`else`	**Check 4, part 2:** the front is not on the first node...
68	`f = f->prev;`	...so front simply moves one node back, using its own `prev` link. No search.
70	`f->data = x;`	Store the new value in the new front node.

Q. What does line 57 check?

If the node after the rear is the front. Then there is no free node left anywhere in the ring: the deque is full after wrapping (Circular Deque Overflow).

Q. Why is Check 2 before Check 3 when Check 3 would also catch that case?

If f == head and r->next == head, then r->next == f as well. Check 2 is placed first only so the message says it was a plain (not wrapped) full deque.

Q. Why must Check 1 come first?

Checks 2 and 3 use `r->next`. If r were NULL that would crash. Check 1 handles the NULL case before anything touches r->next.

Q. Why do the full checks come before Check 4?

Check 4 moves f. The full test has to happen before the move; otherwise f would move onto the rear's node and overwrite a value.

Q. When f is on head, `f->prev` and `head->prev` are the same node. Why write Check 4 in two parts?

Both give the last node. The two parts make the **wrap** (first node back to last node) a separate, visible check, the same way dpushRear writes its wrap. It also matches the lab rule of distinct checks.

Q. Why is this O(1) here but was a search in the singly deque?

Every node stores `prev`, so the node before f is read directly. A singly linked node only has `next`, so the old code had to walk from head to find it.

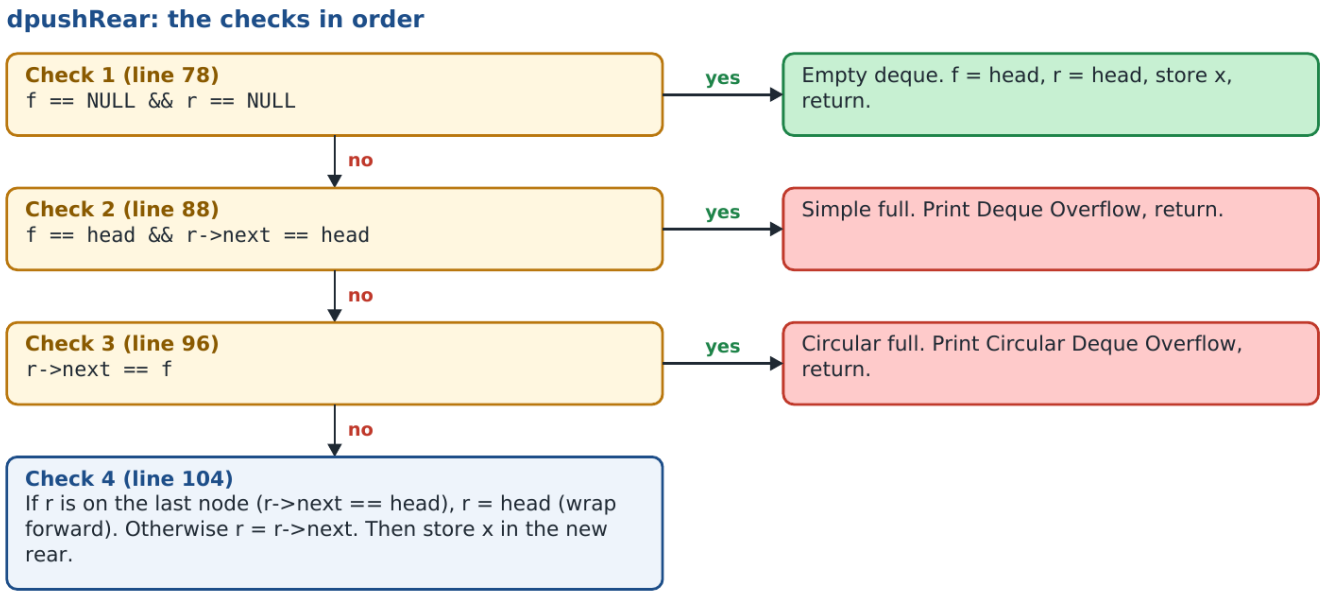
Q. Why is f passed by reference but head by value?

f is moved and the menu's `dfront[dcur]` must see the new position. head never changes, it is only read.

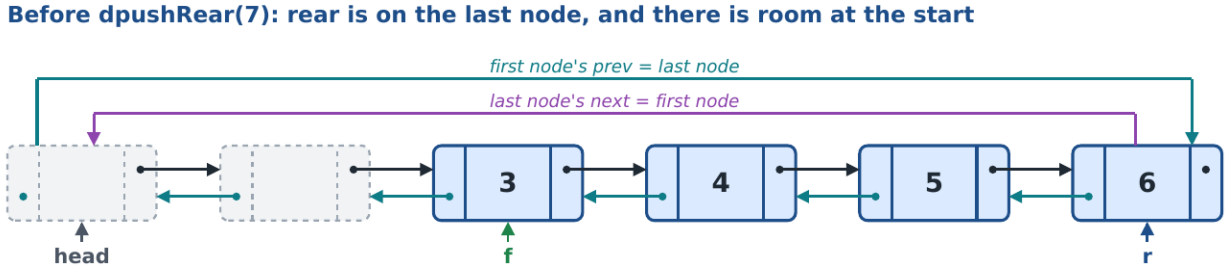
6.2 dpushRear: insert at the rear

What it does: puts a new value **after** the current rear. The rear moves **one node forward**. It is exactly the circular-queue insert, with deque messages.

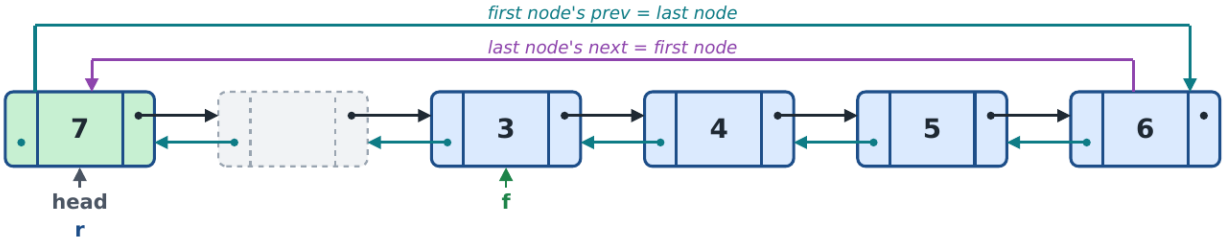
Parameter	Passed how	Why the function needs it
head	by value	Starts an empty deque and is used to test 'rear is on the last node' (r->next == head) and to wrap r back to the first node.
f	by reference	Set on the first insert (f = head). Read in the full checks.
r	by reference	The rear moves forward. The caller must see the new rear.
x	by value (int)	The value to insert.



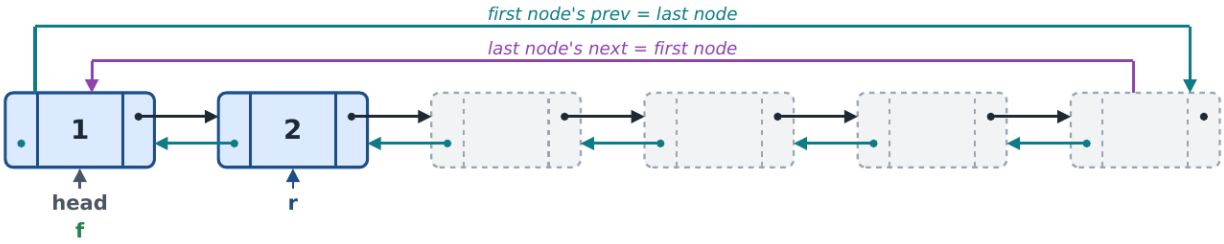
Pictures of Check 4



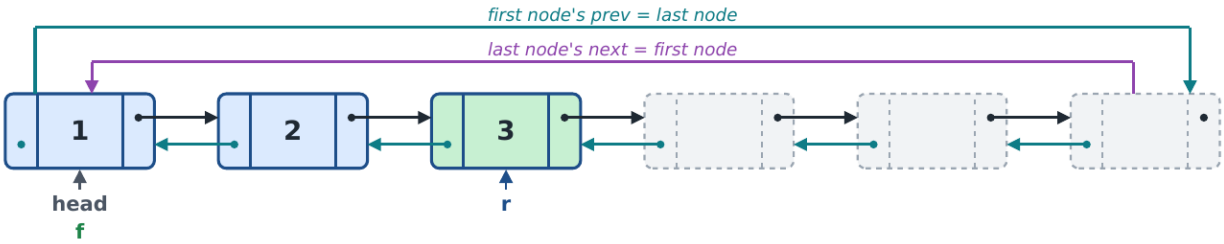
After: r->next == head, so r = head and 7 is stored there. Front to rear reads 3 4 5 6 7



Normal case, before dpushRear(3)



After: r = r->next, store 3



Checks 1, 2 and 3 are the same as in dpushFront (same pictures as section 6.1).

Line by line

Line	Code (deque.cpp)	What this line does, and why
74	`void func::dpushRear(node *head, node *&f, node *&r, int x)`	Function header: insert x at the rear.
78	`if (f == NULL && r == NULL)`	**Check 1:** the deque is empty.
80	`f = head;`	Front goes to the first node...
81	`r = head;`	...and rear too: one value is both ends.
82	`r->data = x;`	Store the value.
83	`return;`	Done.
88	`if (f == head && r->next == head)`	**Check 2 (simple full):** front on the first node and rear on the last node.
90	`cout << "Deque Overflow" << endl;`	Full message.

Line	Code (deque.cpp)	What this line does, and why
91	`return;`	Do not insert.
96	`if (r->next == f)`	**Check 3 (circular full):** front = rear + 1, the ring is full after wrapping.
98	`cout << "Circular Deque Overflow" << endl;`	Full message for the wrapped case.
99	`return;`	Do not insert.
104	`if (r->next == head)`	**Check 4, part 1:** the node after rear is head, so rear is on the **last** node...
105	`r = head;`	...so rear wraps forward to the first node. (Same node as r->next; written out so the wrap is easy to see.)
106	`else`	**Check 4, part 2:** otherwise...
107	`r = r->next;`	...rear simply moves one node forward.
109	`r->data = x;`	Store the value in the new rear node.

Q. What is the difference between dpushRear and dpushFront?

dpushRear moves r **forward** with `next`. dpushFront moves f **backward** with `prev`. Both are one step, and the first three checks are identical.

Q. Line 105 does `r = head`, but r->next is already head. Why write it?

It is the same node. Writing it as a separate check makes the wrap from the last node to the first node visible, which is one of the required checks.

Q. Is dpushRear different from the circular queue's cpush?

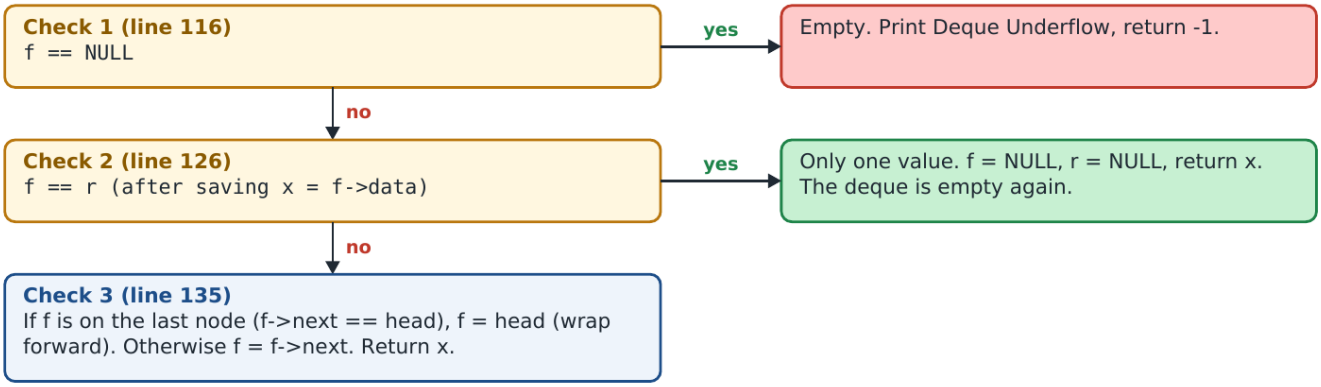
No, the logic is the same. Only the messages say Deque instead of Queue. The `prev` links are not used here at all.

6.3 dpopFront: delete from the front

What it does: removes the value at the front and returns it. The front moves **one node forward**. On an empty deque it prints **Deque Underflow** and returns -1.

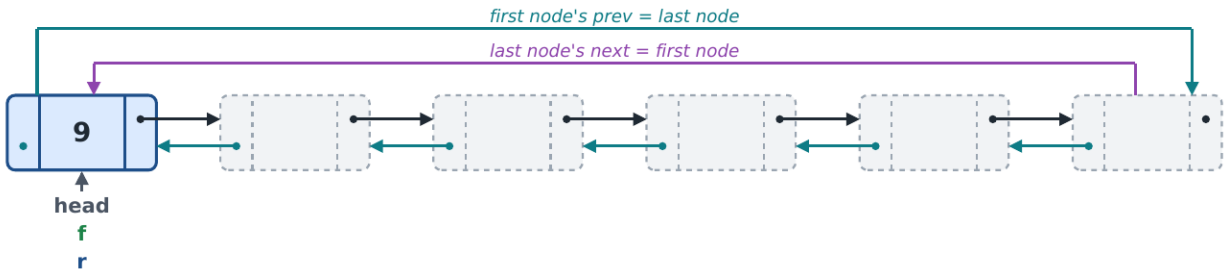
Parameter	Passed how	Why the function needs it
head	by value	Used by Check 3 to wrap the front from the last node to the first node.
f	by reference	The front moves forward, or becomes NULL when the last value is removed.
r	by reference	Becomes NULL when the last value is removed.
return value	int	The value removed, or -1 when the deque was empty.

dpopFront: the checks in order

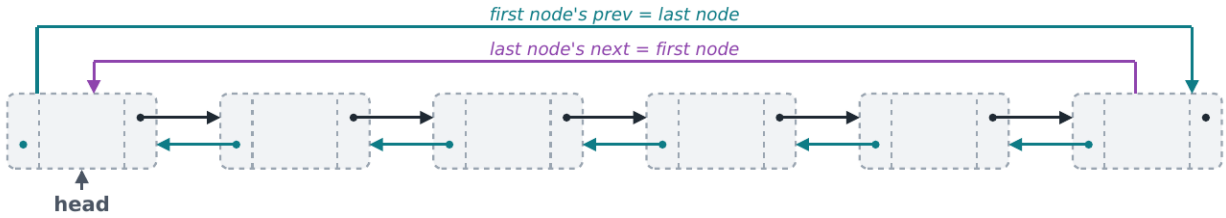


Pictures

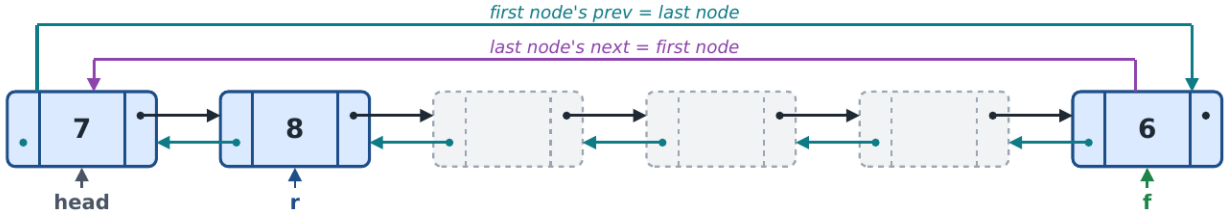
Check 2, before: one value (f == r)



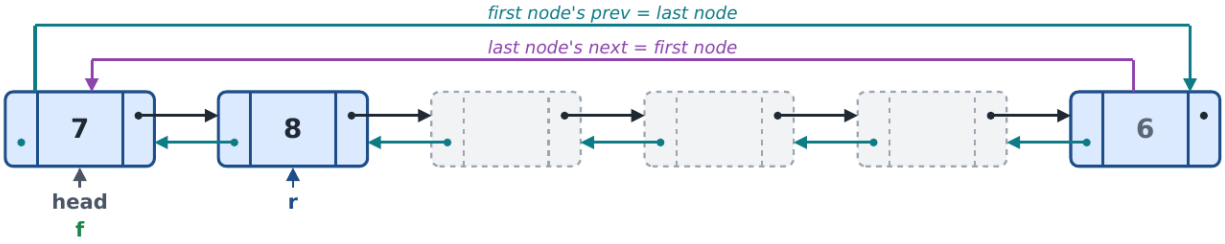
After dpopFront (returns 9): f = NULL, r = NULL. The next insert starts at the first node again



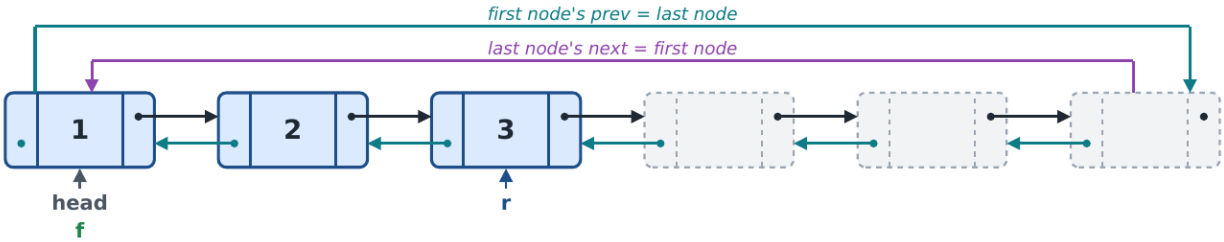
Check 3, before: front on the last node (f->next == head). Reads 6 7 8



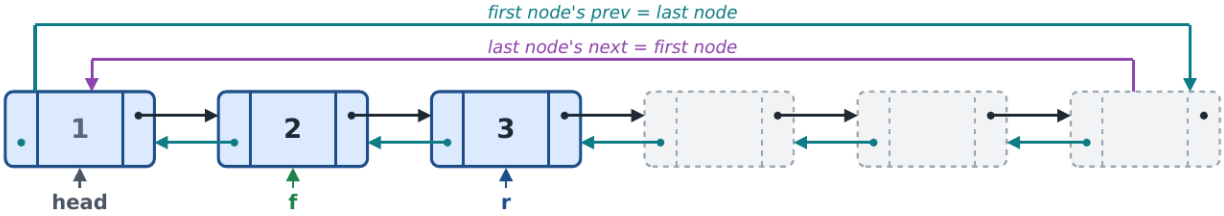
After (returns 6): f = head. The old node keeps the 6 but is now free



Normal case, before dpopFront



After (returns 1): f = f->next



A deleted value is not erased. Its node is simply outside the f..r part, so the next insert overwrites it.

Line by line

Line	Code (deque.cpp)	What this line does, and why
113	`int func::dpopFront(node *head, node *&f, node *&r)`	Function header: delete from the front, return the value.
116	`if (f == NULL)`	**Check 1:** nothing to delete.
118	`cout << "Deque Underflow" << endl;`	Underflow message.
119	`return -1;`	-1 means 'no value'.
122	`int x = f->data;`	Save the front value **before** moving f, so it can be returned.
126	`if (f == r)`	**Check 2:** front and rear on the same node means only one value.
128	`f = NULL;`	The deque becomes empty...
129	`r = NULL;`	...both pointers NULL, exactly the state Check 1 of the inserts looks for.

Line	Code (deque.cpp)	What this line does, and why
130	`return x;`	Return the removed value.
135	`if (f->next == head)`	**Check 3, part 1:** the front is on the last node...
136	`f = head;`	...so it wraps forward to the first node.
137	`else`	**Check 3, part 2:** otherwise...
138	`f = f->next;`	...the front moves one node forward.
139	`return x;`	Return the removed value.

Q. Why is the value saved before the checks that move f?
After f moves, the old front node is no longer known, so its value would be lost.

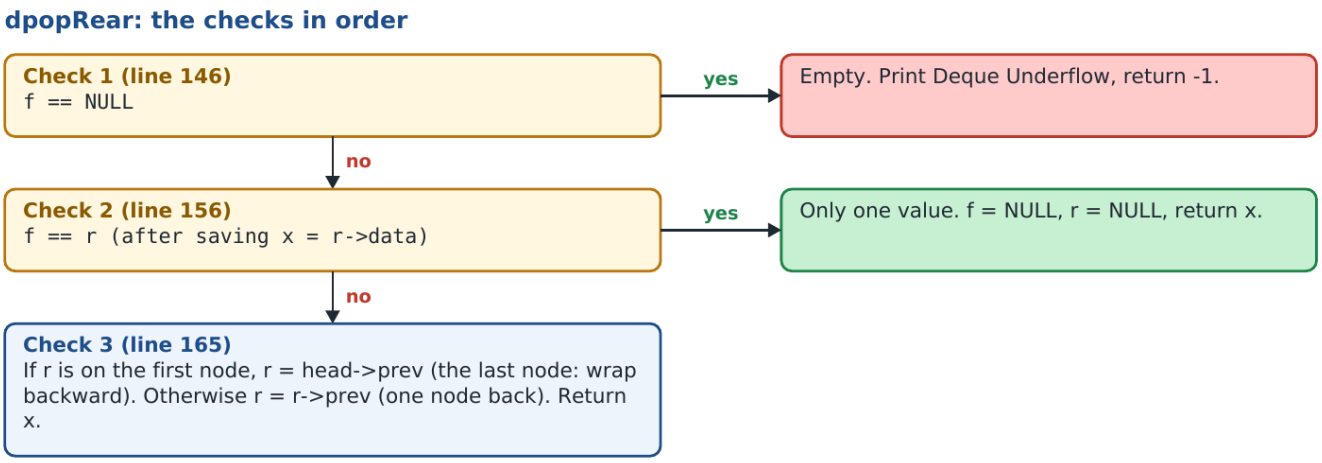
Q. Why set both f and r to NULL when the last value goes?
So the deque is clearly empty: Check 1 of dpushFront/dpushRear tests `f == NULL && r == NULL` and restarts at the first node.

Q. The menu checks `f == NULL` before calling dpopFront. Why does the function check again?
The function is also called by other functions (palindrome, remove duplicates). Its own check makes it safe everywhere. The menu check only stops it printing 'Deleted: -1'.

6.4 dpopRear: delete from the rear

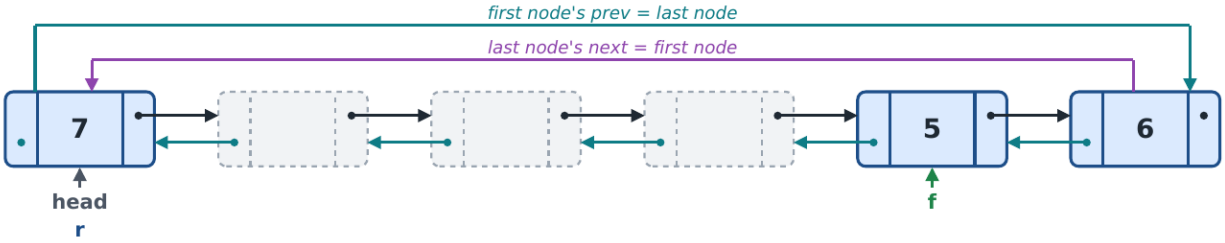
What it does: removes the value at the rear and returns it. The rear moves **one node back**. This is the second backward move, and like dpushFront it uses `prev`.

Parameter	Passed how	Why the function needs it
head	by value	Needed to test 'rear is on the first node' and to reach the last node in one step (`head->prev`).
f	by reference	Becomes NULL when the last value is removed.
r	by reference	The rear moves back, or becomes NULL.
return value	int	The value removed, or -1 when empty.

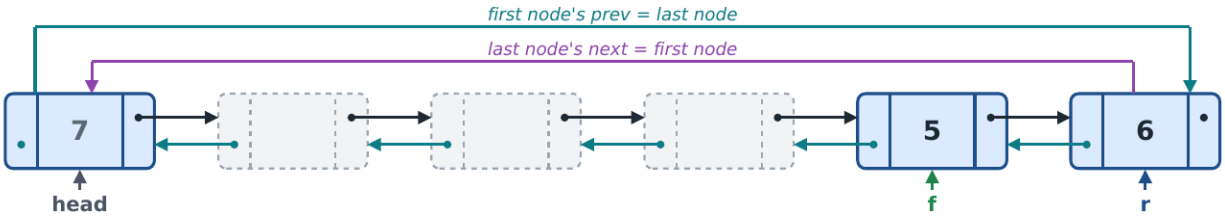


Pictures of Check 3

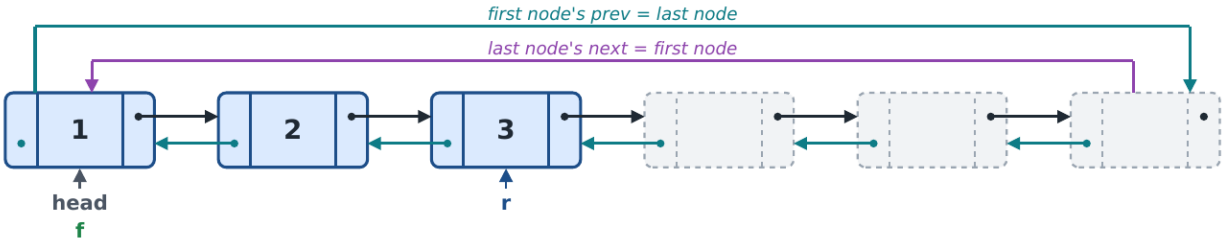
Before dpopRear: rear is on the first node. Reads 5 6 7



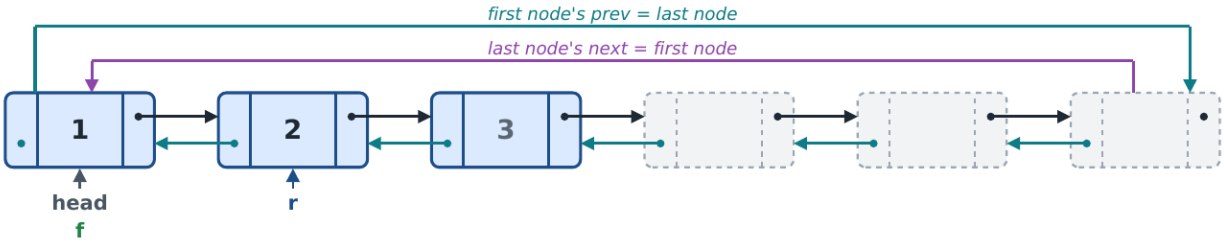
After (returns 7): line 166 puts r on the last node (head->prev). Reads 5 6



Normal case, before dpopRear



After (returns 3): r = r->prev



Line by line

Line	Code (deque.cpp)	What this line does, and why
143	`int func::dpopRear(node *head, node *&f, node *&r)`	Function header: delete from the rear, return the value.
146	`  if (f == NULL)`	**Check 1:** nothing to delete.

Line	Code (deque.cpp)	What this line does, and why
148	`    cout << "Deque Underflow" << endl;`	Underflow message.
149	`    return -1;`	-1 means 'no value'.
152	`  int x = r->data;`	Save the rear value before r moves.
156	`  if (f == r)`	**Check 2:** only one value left.
158	`    f = NULL;`	The deque becomes empty…
159	`    r = NULL;`	…both pointers NULL.
160	`    return x;`	Return the removed value.
165	`  if (r == head)`	**Check 3, part 1:** the rear is on the first node, so the node before it is the **last** node (wrap backward).
166	`    r = head->prev;     // head's prev is the last node`	The first node's `prev` is the last node, so the rear wraps back to it in one step.
167	`  else`	**Check 3, part 2:** otherwise…
168	`    r = r->prev;`	…the rear moves one node back with its own `prev` link.
169	`  return x;`	Return the removed value.

Q. How does dpopRear move back without a search?

Every node has a `prev` pointer, so the node before r is `r->prev`. In a singly linked list r would need a search from head, because a node there has no pointer to the previous node.

Q. Which two operations move a pointer backward?

dpushFront (f moves back) and dpopRear (r moves back). Both use the same two-part check: on head, go to `head->prev` (the last node); otherwise one step back with `prev`.

Q. How can the deque act as a stack?

Insert with dpushRear and delete with dpopRear: both work at the rear, so the last value in is the first out. Q8 (brackets) does exactly this.

7. Display, display backward, count, search and reverse

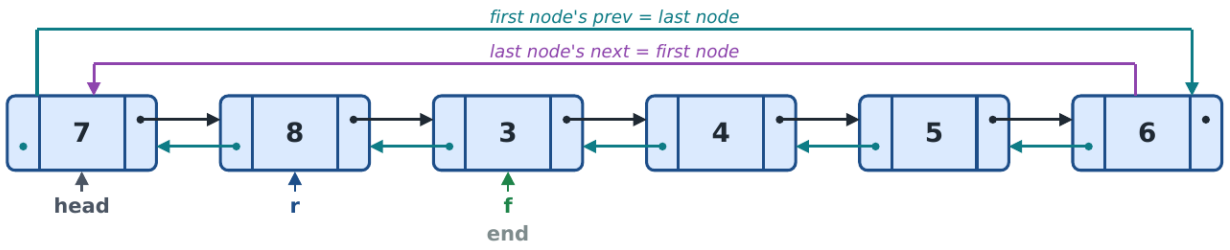
These five functions only **read** the deque or swap values. They never move f or r, so f and r are passed **by value**.

7.1 ddisplay

What it does: prints the values from the front to the rear, going through the last node into the first node when the deque has wrapped.

Parameter	Passed how	Why the function needs it
f	by value	Where printing starts.
r	by value	Where printing ends. Only read.

A full wrapped deque: end = r->next is the front itself



With a normal while (p != end) loop, p == end at the very start, so nothing would print. do...while prints first and tests afterwards, so the loop runs all the way round once: 3 4 5 6 7 8.

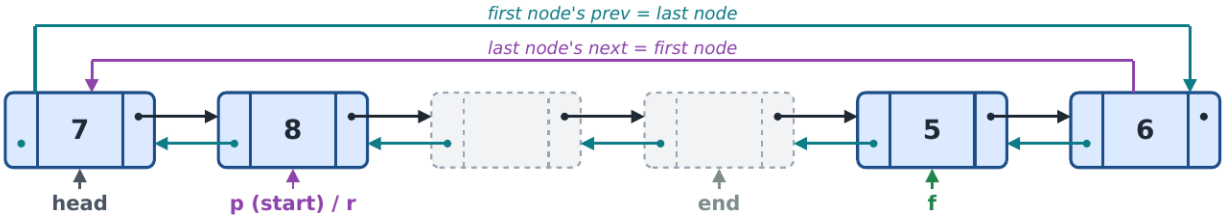
Line	Code (deque.cpp)	What this line does, and why
173	`void func::ddisplay(node *f, node *r)`	Function header.
175	`  if (f == NULL)`	Empty deque?
177	`    cout << "empty" << endl;`	Print the word empty...
178	`    return;`	...and stop.
180	`  node *p = f;`	p walks from the front.
181	`  node *end = r->next;    // node after rear (front itself when the deque is full)`	Stop when p reaches the node after the rear. When the deque is full this is the front itself.
182	`  do                      // do...while so a full deque is still printed once`	Start of do...while: the body always runs at least once.
184	`    cout << p->data << " ";`	Print the value at p.
185	`    p = p->next;`	Move forward. From the last node this goes to the first node by itself (ring).
186	`  } while (p != end);`	Repeat until p has passed the rear.
187	`  cout << endl;`	New line after the values.

7.2 ddisplayBack (rear to front)

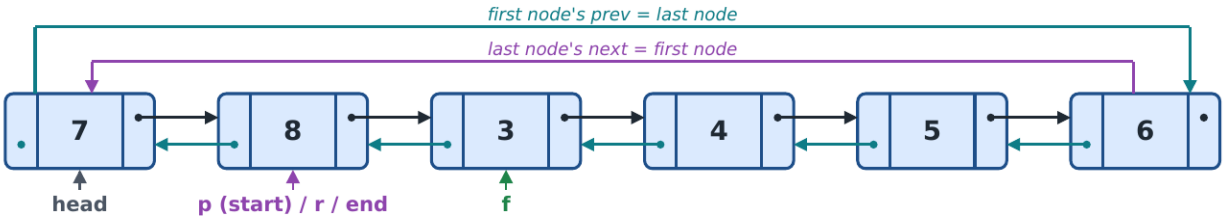
What it does: prints the values from the **rear to the front** by following `prev`. This function is new in the doubly linked version: a singly linked list cannot walk backward without searching.

Parameter	Passed how	Why the function needs it
f	by value	Where printing ends. Only read (its `prev` is the stop node).
r	by value	Where printing starts.

Wrapped deque 5 6 7 8: p starts at r and follows prev; end = f->prev. Prints 8 7 6 5



Full deque: end = f->prev is the rear itself, so do...while is needed. Prints 8 7 6 5 4 3



ddisplayBack is ddisplay mirrored: start at r instead of f, use prev instead of next, and stop at f->prev instead of r->next.

Line	Code (deque.cpp)	What this line does, and why
191	`void func::ddisplayBack(node *f, node *r)`	Function header.
193	`  if (f == NULL)`	Empty deque?
195	`    cout << "empty" << endl;`	Print the word empty...
196	`    return;`	...and stop.
198	`  node *p = r;`	p walks from the **rear**.
199	`  node *end = f->prev;    // node before front (rear itself when the deque is full)`	Stop when p reaches the node **before** the front. When the deque is full this is the rear itself.
200	`  do`	do...while: the body runs once before the test, so a full deque still prints.
202	`    cout << p->data << " ";`	Print the value at p.
203	`    p = p->prev;`	Move **backward**. From the first node this goes to the last node by itself (the list is circular).
204	`  } while (p != end);`	Repeat until p has passed the front.

Line	Code (deque.cpp)	What this line does, and why
205	`  cout << endl;`	New line after the values.

7.3 dcount and dsearch

Both walk from f to r with the same do...while loop as ddisplay. dcount adds 1 for every node and returns the total. dsearch returns 1 as soon as it finds x, otherwise 0. They are used by the questions (split, sort, reverse, palindrome, duplicates, intersection).

Line	Code (deque.cpp)	What this line does, and why
209	`int func::dcount(node *f, node *r)`	Function header: returns the number of values.
211	`  if (f == NULL)`	Empty deque...
212	`    return 0;`	...has 0 values.
213	`  int c = 0;`	Counter.
214	`  node *p = f;`	Start at the front.
215	`  node *end = r->next;`	Stop after the rear.
216	`  do`	do...while for the full-ring case.
218	`    c++;`	Count this node.
219	`    p = p->next;`	Move forward (wraps by itself).
220	`  } while (p != end);`	Until past the rear.
221	`  return c;`	Return the count.

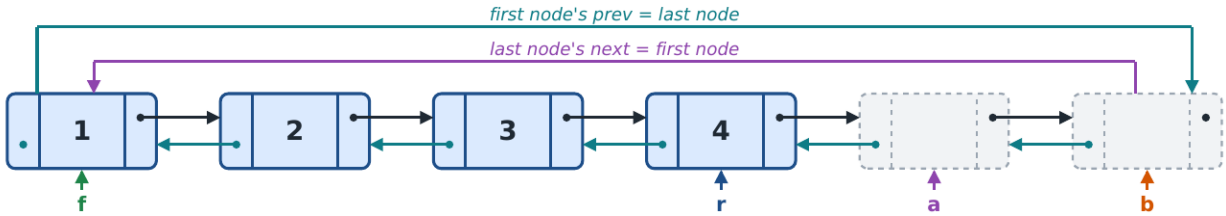
Line	Code (deque.cpp)	What this line does, and why
225	`int func::dsearch(node *f, node *r, int x)`	Function header: 1 if x is found, 0 if not (no bool is used).
227	`  if (f == NULL)`	Empty deque...
228	`    return 0;`	...contains nothing.
229	`  node *p = f;`	Start at the front.
230	`  node *end = r->next;`	Stop after the rear.
231	`  do`	do...while for the full-ring case.
233	`    if (p->data == x)`	Is this the value?
234	`      return 1;`	Found: stop at once.
235	`    p = p->next;`	Move forward.
236	`  } while (p != end);`	Until past the rear.
237	`  return 0;`	Went through all values: not found.

7.4 dreverse

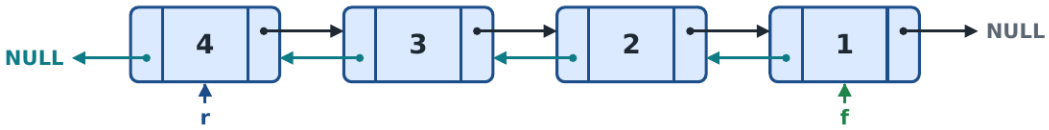
What it does: reverses the deque by **turning the links**. The values never move. Every node from the front to the rear swaps its `next` and `prev`, then the old rear becomes the new front and the old front becomes the new rear. Finally the two ends are joined to the free nodes again so the ring stays closed.

Parameter	Passed how	Why the function needs it
f	by reference (`node *&f`)	The front changes: it becomes the old rear. The caller (`dfront[dcur]`) must see the new front.
r	by reference (`node *&r`)	The rear changes: it becomes the old front.

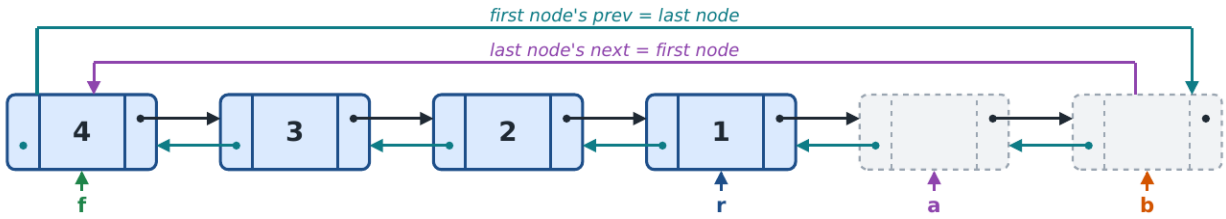
Before: f on 1, r on 4. a = r->next is the first free node, b = f->prev is the last free node



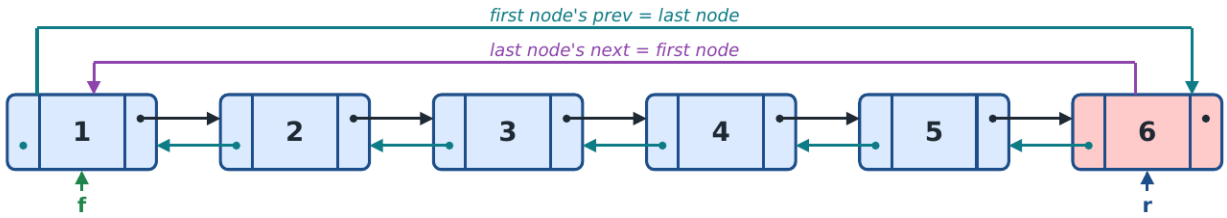
After the for loop: next and prev swapped in all four nodes, so with next it reads 4 3 2 1. The ends are not joined to a and b yet



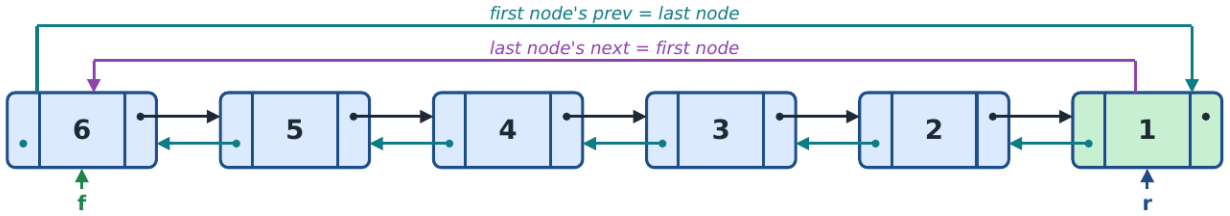
After f and r swap and the ends are joined: f = old r, r = old f. Front to rear reads 4 3 2 1



Full deque: a == r, so there is no free node. Only the links turn and f and r swap



Full deque after dreverse: reads 6 5 4 3 2 1, the ring is still closed



Line	Code (deque.cpp)	What this line does, and why
241	`void func::dreverse(node *&f, node *&r)`	Function header. f and r by reference because they change.

244-247	`  if (f == NULL)`
	`    cout << "Deque is empty" << endl;`
	`    return;`
		Check 1: an empty deque has nothing to reverse.

251-252	`  if (f == r)` `    return;`	**Check 2:** one value, reversing changes nothing.
254	`  int n = dcount(f, r);   // values in the deque`	How many nodes hold values (the loop turns exactly these nodes).
255	`  node *a = r->next;      // first free node after the rear`	Remember a, the first free node after the rear. (When the deque is full this is the front itself.)
256	`  node *b = f->prev;      // last free node before the front`	Remember b, the last free node before the front.
259	`  node *p = f;`	p walks from the front to the rear.
260	`  node *q;`	q keeps the old next while we change it.
261	`  for (int i = 0; i < n; i++)`	Visit each of the n nodes once.
263	`    q = p->next;        // remember the old next`	Save the old next: after the swap we still need it to go forward.
264	`    p->next = p->prev;`	The new next is the old prev.
265	`    p->prev = q;`	The new prev is the old next.
266	`    p = q;`	Move to the next node of the **old** order.
270	`  node *t = f;`	Keep the old front.
271	`  f = r;`	The old rear is now the front.
272	`  r = t;`	The old front is now the rear.
276	`  if (a != r)`	**Check 3:** a == r means the deque filled the whole ring, so nothing is left to join. Otherwise do the next four lines.
278	`    f->prev = b;`	The new front points back to b, the last free node.
279	`    b->next = f;`	b points forward to the new front.
280	`    r->next = a;`	The new rear points forward to a, the first free node.
281	`    a->prev = r;`	a points back to the new rear. The ring is closed again.

Q. Why do display, count and search use do...while and not while?

When the deque is full, `r->next` is the front, so `end == f` at the start. A while loop would stop immediately; do...while runs the body once first.

Q. Why are f and r by value in display, count and search?

They never move the pointers. They only read values. dreverse is different: it changes f and r, so they are references.

Q. Does dreverse move the data?

No. The values stay in their nodes. Only the next and prev links are turned and f and r swap roles, so the old rear (which holds the last value) is now the front.

Q. Why does dreverse need a and b?

After turning the links the two ends still point to the wrong neighbours. a (first free node after the rear) and b (last free node before the front) are saved first so the ends can be joined to them again.

Q. What if the deque is full?

Then a is the old front, so after the swap a == r and Check 3 skips the joining. The ring is already closed, so only the links turn and f and r swap.

Q. Is reversing by links faster than swapping data?

Both are O(n). Turning links does not copy any values, which matters when each node holds a big record.

Q. Why does ddisplayBack stop at f->prev and not at f?

The front itself must be printed. The loop stops at the node **after** the front in backward order, which is f->prev.

8. The question functions (Q1 to Q10)

Each question works on the deque that the menu passes in. Questions that produce new deques (Q1, Q2, Q7, Q10) write into new array slots, so the result becomes a real deque you can select. All inserts go through `dpushRear` (or `dpushFront`), so the overflow checks always apply.

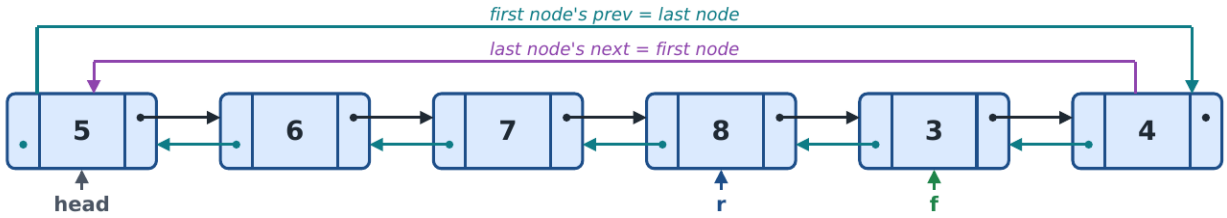
8.1 Q1: splitDeque (split into two deques)

What it does: makes two new deques from one: the first gets the first half of the nodes, the second gets the rest. With 6 nodes each half has **3 nodes**, so each new deque overflows after 3 values. The original deque is not changed.

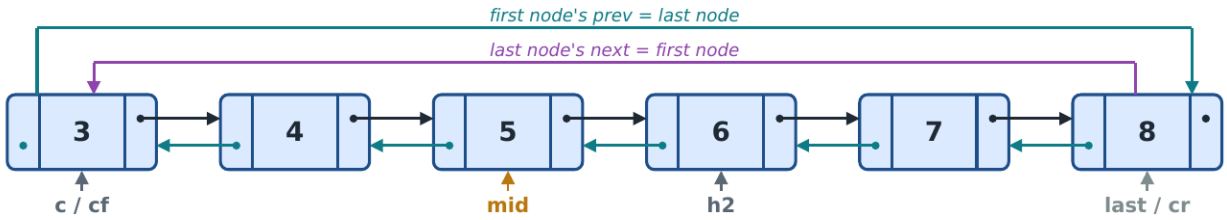
How: copy the deque into a new doubly circular list, then **cut the copy** into two lists by changing four links: two `next` links and two `prev` links. No smaller list is built.

Parameter	Passed how	Why the function needs it
head	by value	First node of the base ring. Used to count its nodes (m).
f, r	by value	Front and rear of the base deque: what to copy and how many values (n).
h1, f1, r1	by reference	**Output:** head, front, rear of the first half. The menu passes `dhead[a]`, `dfront[a]`, `drear[a]`.
h2, f2, r2	by reference	**Output:** head, front, rear of the second half (slot b).
return value	int	1 = split done, 0 = fewer than 2 nodes, cannot split.

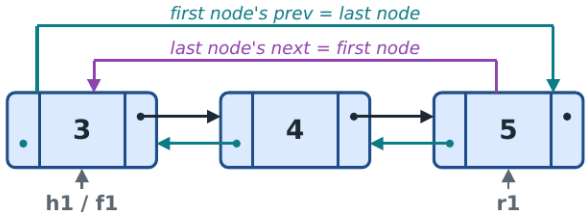
Base deque (wrapped), reads 3 4 5 6 7 8. It is only read, never changed



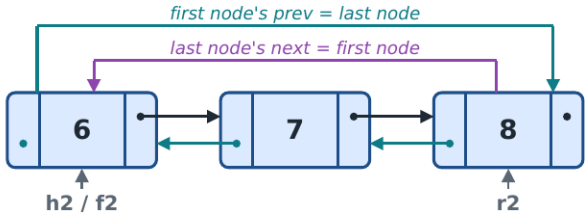
The copy c: copyDeque starts at the first node, so it is not wrapped. m = 6, k = 3



List 1: mid->next = c and c->prev = mid. Front 3, rear 5, full (3 of 3 nodes)



List 2: last->next = h2 and h2->prev = last. Front 6, rear 8, full



Four links changed: mid->next and c->prev close list 1 in both directions; last->next and h2->prev close list 2.

> **Fewer values than nodes**
> Base deque 3 4 5 6 (n = 4): list 1 holds 3 4 5 (full) and list 2 holds 6 with two free nodes. Base deque 3 4 (n = 2, n <= k): list 1 holds 3 4 and list 2 is empty (f2 = r2 = NULL).

Line	Code (deque.cpp)	What this line does, and why
289	`int func::splitDeque(node *head, node *f, node *r, node *&h1, node *&f1, node *&r1, node *&h2, node *&f2, node *&r2)`	Function header: 9 parameters because each deque needs 3 pointers (head, front, rear): 1 input deque and 2 output deques.
292	`  int m = 0;`	m will be the number of **nodes** in the base ring.
293	`  node *q = head;`	q walks round the ring from head.
294	`  do`	do...while: count head first, then keep going.
296	`    m++;`	Count one node.
297	`    q = q->next;`	Move on.
298	`  } while (q != head);`	Stop when we are back at head: every node counted once.
299	`  if (m < 2)`	A ring with fewer than 2 nodes cannot be cut in two...
300	`    return 0;`	...so report failure (the menu prints a message).
301	`  int n = dcount(f, r);`	n = number of **values** in the base deque.
304	`  node *c = NULL, *cf = NULL, *cr = NULL;`	Head, front and rear of the copy.
305	`  createList(c);`	Build a new full-size doubly circular list for the copy.
306	`  copyDeque(f, r, c, cf, cr);`	Copy all values into it (front to rear, starting at the copy's first node).

Line	Code (deque.cpp)	What this line does, and why
309	`node *last = c->prev;`	The last node of the copy, where list 2 will end. Because the list is doubly circular, the first node's `prev` **is** the last node: no loop is needed.
312	`int k = (m + 1) / 2;`	k = nodes in the first half. m = 6 gives 3; m = 7 would give 4.
313	`node *mid = c;`	Find the k-th node: the last node of list 1.
314	`for (int i = 1; i < k; i++)`	Move k - 1 times...
315	`mid = mid->next;`	...one node each time.
317	`h1 = c;`	Ring 1 starts at the copy's first node.
318	`h2 = mid->next;`	Ring 2 starts at the node after mid. Saved **before** the link is changed.
319	`mid->next = c;  // first ring: last node -> first node`	**Cut 1 (next):** mid now points forward to list 1's first node, closing list 1 forward.
320	`c->prev = mid;  // first node -> last node`	**Cut 1 (prev):** list 1's first node points back to mid, closing list 1 backward.
321	`last->next = h2;  // second ring: last node -> first node`	**Cut 2 (next):** last now points forward to list 2's first node, closing list 2 forward.
322	`h2->prev = last;  // first node -> last node`	**Cut 2 (prev):** list 2's first node points back to last, closing list 2 backward.
325	`f1 = cf;`	Ring 1's front is the copy's front.
326	`if (n <= k)`	Do all values fit in the first half?

328-330	`r1 = cr;`
	`f2 = NULL;`
	`r2 = NULL;`
		Yes: list 1's rear is the copy's rear, and list 2 is empty.

332	`else`	No: values spill into the second half...

334-336	`r1 = mid;`
	`f2 = h2;`
	`r2 = cr;`
		...list 1 is full (rear = mid), list 2 starts at h2 and ends at the copy's rear.

338	`return 1;`	Split done.

Q. Why is the copy made first?

So the original deque is not changed. The copy is cut, not the original.

Q. Why are last and mid found before any link is changed?

After `mid->next = c` a walk forward from c would go round list 1 forever, and after `c->prev = mid` the step `c->prev` would no longer give the copy's last node. So both nodes are located first.

Q. Why must the two prev links be changed as well?

Without `c->prev = mid`, list 1's first node would still point back into list 2. Then dpushFront on list 1 (which uses `head->prev`) would wrap into list 2's nodes and overwrite its values. The same holds for `h2->prev = last`.

Q. When does each half overflow?

When its own nodes are used up: list 1 has k = 3 nodes and list 2 has m - k = 3 nodes. The 4th insert gives Deque Overflow.

Q. Why does the function return int and not bool?

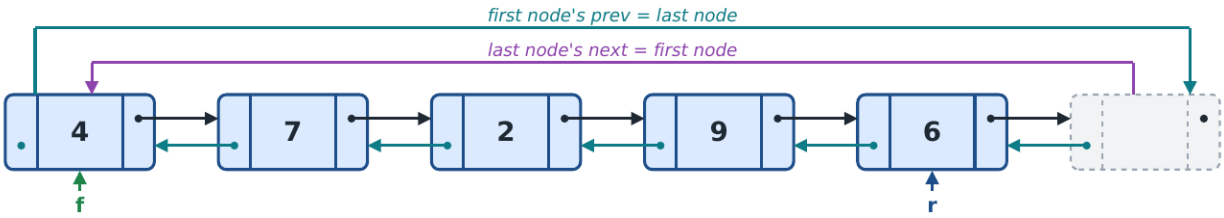
The rules say no bool. 1 means done, 0 means it could not split.

8.2 Q2: devenOdd (even and odd deques)

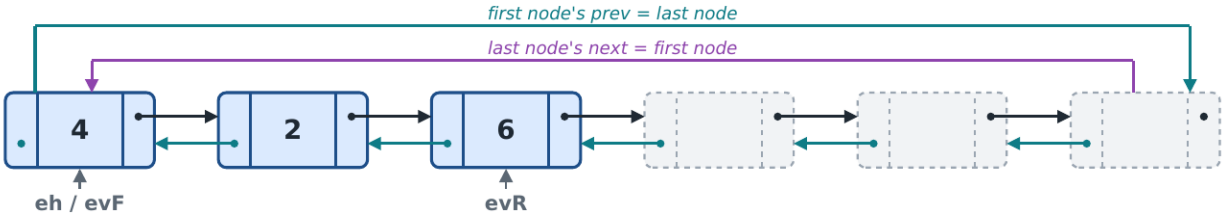
What it does: walks the deque from front to rear once. Every even value is inserted at the rear of the even deque, every odd value at the rear of the odd deque. The original is not changed.

Parameter	Passed how	Why the function needs it
f, r	by value	The source deque. Only read.
eh	by value	Head of the even deque's ring (dpushRear needs it for the first value).
evF, evR	by reference	Front and rear of the even deque. Set by the inserts, so the menu must see them.
oh, odF, odR	same as above	The same three things for the odd deque.

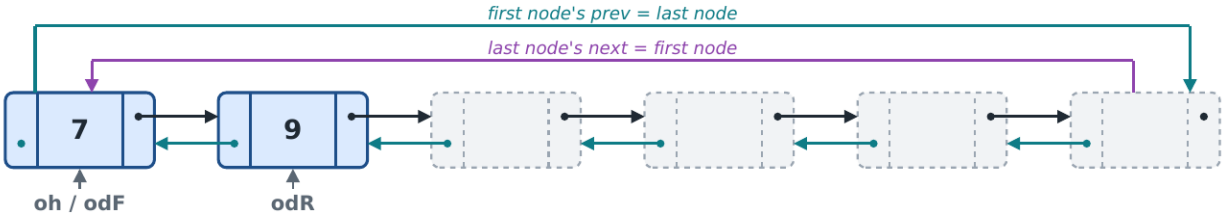
Source deque: 4 7 2 9 6



Even deque: 4 2 6 (original order kept)



Odd deque: 7 9



Line	Code (deque.cpp)	What this line does, and why
342	`void func::devenOdd(node *f, node *r, node *eh, node *&evF, node *&evR, node *oh, node *&odF, node *&odR)`	Function header.

Line	Code (deque.cpp)	What this line does, and why

344-347	`evF = NULL;`
	`evR = NULL;`
	`odF = NULL;`
	`odR = NULL;`
		Both new deques start empty, so the first insert into each uses Check 1 (f = r = head).

348	`if (f == NULL)`	Empty source...
349	`return;`	...nothing to do.
350	`node *p = f;`	Walk from the front.
351	`node *end = r->next;`	Stop after the rear.
352	`do`	do...while (works for a full ring).
354	`if (p->data % 2 == 0)`	Remainder 0 when divided by 2: the value is even.
355	`dpushRear(eh, evF, evR, p->data);`	Insert it at the rear of the even deque (base insert, with its checks).
356	`else`	Otherwise it is odd...
357	`dpushRear(oh, odF, odR, p->data);`	...insert it at the rear of the odd deque.
358	`p = p->next;`	Next value.
359	`} while (p != end);`	Until past the rear.

Q. Why are the result pointers set to NULL first?
So both new deques start empty and the first dpushRear takes the 'empty' path.

Q. Why dpushRear and not dpushFront?
Inserting at the rear keeps the values in their original order.

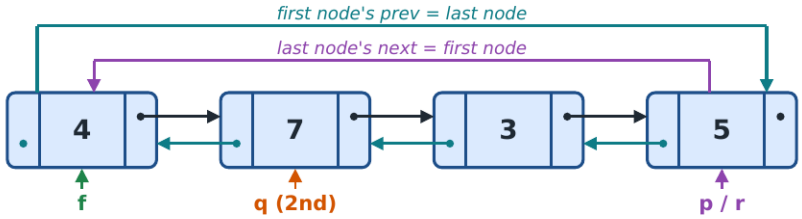
Q. Where do the new deques come from?
The menu creates them in slots dnum and dnum + 1 with createList before calling devenOdd.

8.3 Q3: dnextGreater (next greater element)

What it does: for every value, prints the first value **after it** that is greater, or -1. Because it is a ring, the search continues past the rear back to the front, and looks at the other n - 1 values.

Parameter	Passed how	Why the function needs it
f, r	by value	The deque to examine. Nothing is changed.

p on 5 (the rear): q wraps to the front (4: no), then 7: yes, so 5 -> 7



Output for 4 7 3 5: 4 -> 7, 7 -> -1, 3 -> 5, 5 -> 7

Line	Code (deque.cpp)	What this line does, and why
363	`void func::dnextGreater(node *f, node *r)`	Function header.
365	`  if (f == NULL)`	Empty deque?
367	`    cout << "Deque is empty" << endl;`	Message...
368	`    return;`	...and stop.
370	`  int n = dcount(f, r);`	Number of values. A ring has no NULL end, so the loops are counted.
371	`  node *p = f;`	p goes through every value, starting at the front.
372	`  for (int i = 0; i < n; i++)`	Outer loop: once for each value.
374	`    int nge = -1;`	Answer for p; stays -1 if nothing greater is found.
375	`    node *q = p;`	q will look at the values after p.
377	`    for (int k = 1; k < n; k++)`	Inner loop: the other n - 1 values.
379	`      if (q == r)`	If q is on the rear...
380	`        q = f;              // rear reached, wrap to front`	...wrap to the front (the free nodes after the rear are skipped).
381	`      else`	Otherwise...
382	`        q = q->next;`	...move forward.
384	`      if (q->data > p->data)`	Greater than p's value?
386	`        nge = q->data;`	Yes: that is the answer...
387	`        break;`	...stop looking.
390	`    cout << p->data << " -> " << nge << endl;`	Print 'value -> answer'.
391	`    p = p->next;`	Next value.

Q. Why `if (q == r) q = f;` instead of just `q = q->next`?

If the deque is not full, the nodes after the rear are free and may hold old values. Jumping from the rear straight to the front makes sure only real values are checked.

Q. Why does the inner loop run n - 1 times?

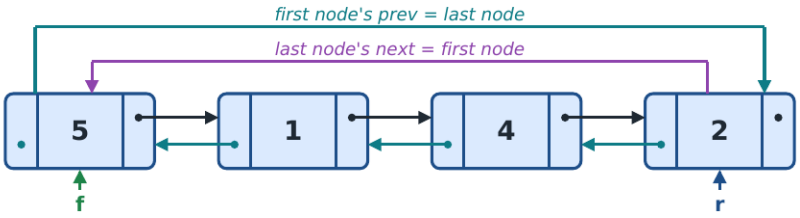
To look at every **other** value once. Running n times would come back to p itself.

8.4 Q5: dsortList (bubble sort)

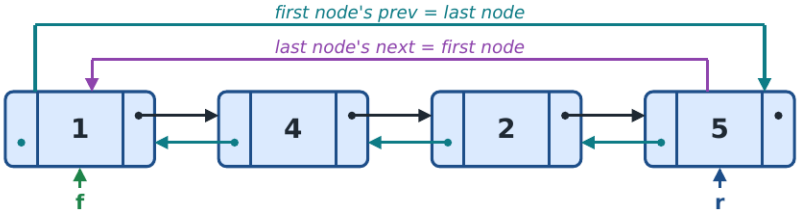
What it does: sorts the values from smallest (front) to largest (rear) with bubble sort. Only the **data** is swapped; f and r stay on the same nodes.

Parameter	Passed how	Why the function needs it
f, r	by value	The deque to sort. The pointers do not move, so no reference.

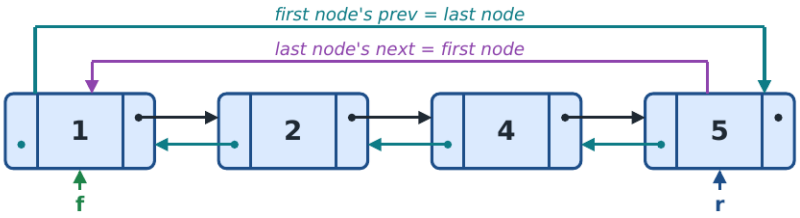
Before: 5 1 4 2



After pass 1 (i = 0): the biggest value 5 has bubbled to the rear



After pass 2: sorted. Pass 3 makes no swap



Line	Code (deque.cpp)	What this line does, and why
396	`void func::dsortList(node *f, node *r)`	Function header.
398	`  int n = dcount(f, r);`	Number of values.
399	`  for (int i = 0; i < n - 1; i++)`	n - 1 passes.
401	`    node *p = f;`	Each pass starts at the front.
402	`    for (int j = 0; j < n - 1 - i; j++)`	Compare neighbours. Each pass needs one compare fewer because the end is already sorted.
404	`      if (p->data > p->next->data)`	Wrong order (bigger value first)?

Line	Code (deque.cpp)	What this line does, and why

406-408	`int t = p->data;`
	`p->data = p->next->data;`
	`p->next->data = t;`
		Swap the two values using t.

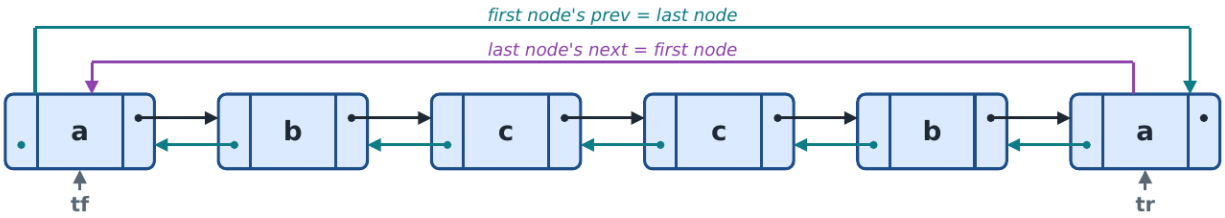
410	`p = p->next;`	Move to the next pair. In a wrapped deque `next` goes from the last node to the first, which is still the correct next value.

8.5 Q6: disPalindrome (palindrome check)

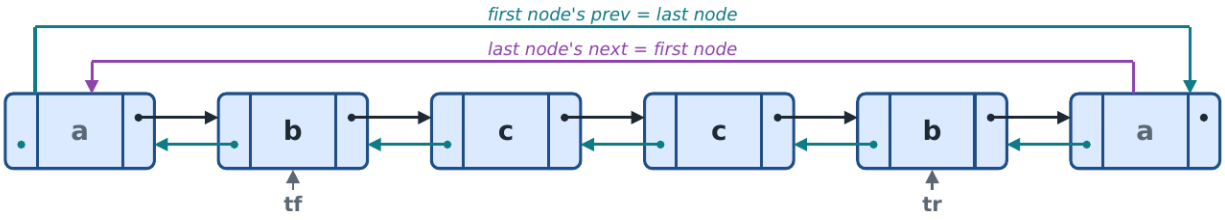
What it does: says whether a word reads the same both ways. It is the **natural deque method**: put every letter in, then take one from **each end** and compare.

Parameter	Passed how	Why the function needs it
s	by value (string)	The word. It works on the temporary ring dtmp with its own local front and rear (tf, tr), so the user's deques are untouched.
return value	int	1 = palindrome, 0 = not.

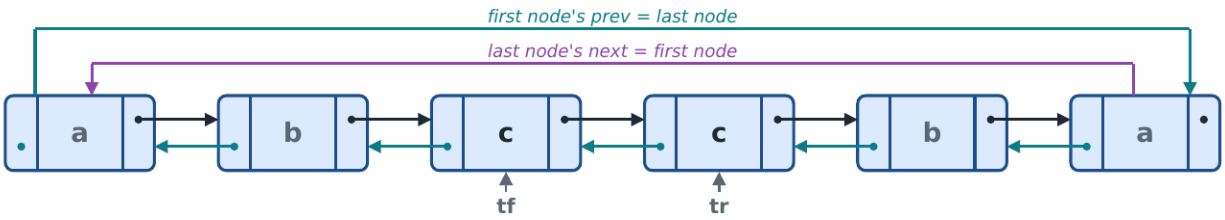
All letters inserted at the rear. Step 1: dpopFront gives a, dpopRear gives a: equal



Step 2: b and b: equal



Step 3: c and c: equal. same = 3 = len / 2, so Palindrome



The menu refuses a word longer than 6 letters, because dtmp has only 6 nodes.

Line	Code (deque.cpp)	What this line does, and why
418	`int func::disPalindrome(string s)`	Function header.
420	`  node *tf = NULL, *tr = NULL;`	Local front and rear for dtmp. They start NULL every call, so the temporary deque is always empty at the start.
421	`  int len = s.length();`	Number of letters.
422	`  for (int i = 0; i < len; i++)`	For every letter...
423	`    dpushRear(dtmp, tf, tr, s[i]);`	...insert it at the rear (the letter is stored as its character code).
425	`  int same = 0;`	Counts matching pairs.
426	`  for (int i = 0; i < len / 2; i++)`	len / 2 pairs. An odd middle letter needs no partner.
428	`    int a = dpopFront(dtmp, tf, tr);`	Take a letter from the front (first, second, ...).
429	`    int b = dpopRear(dtmp, tf, tr);`	Take a letter from the rear (last, second-last, ...).
430	`    if (a == b)`	Same letter?
431	`      same++;`	Count the matching pair.
433	`  if (same == len / 2)`	Every pair matched?
434	`    return 1;`	Palindrome.
435	`  return 0;`	Not a palindrome.

Q. Why is a deque good for palindromes?

It gives both ends directly: compare front with rear, then move inward. A stack or queue alone gives only one end.

Q. Why are the two pops written as separate lines (a and b)?

So the order is clear and fixed: first from the front, then from the rear.

Q. Why is a one-letter word a palindrome?

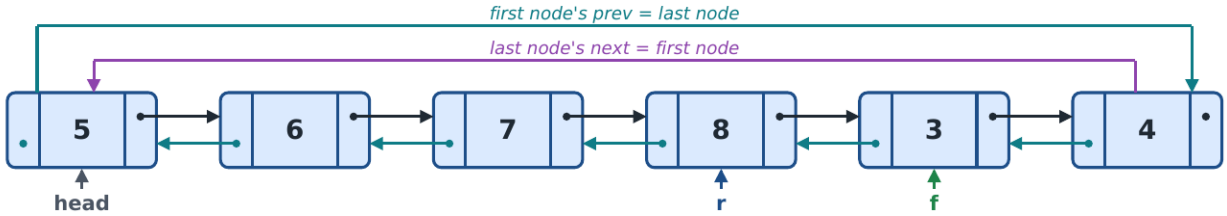
len / 2 = 0 pairs, same = 0 = len / 2, so it returns 1.

8.6 Q7: copyDeque (copy)

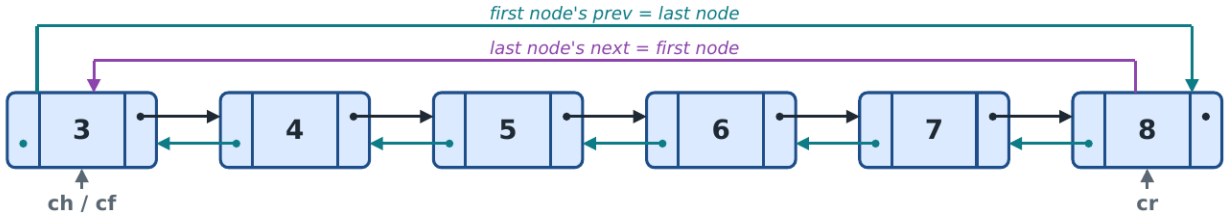
What it does: inserts every value of a deque, front to rear, into another deque. Q1 also uses it.

Parameter	Passed how	Why the function needs it
f, r	by value	The source deque. Only read.
ch	by value	Head of the destination ring (dpushRear needs it for the first value).
cf, cr	by reference	Front and rear of the copy. Set by the inserts; reset to NULL first so the copy starts empty.

Original (wrapped): reads 3 4 5 6 7 8



Copy: same values in the same order, starting at the copy's first node



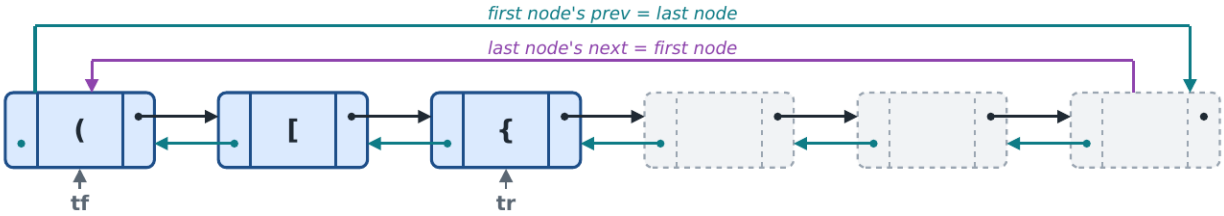
Line	Code (deque.cpp)	What this line does, and why
439	`void func::copyDeque(node *f, node *r, node *ch, node *&cf, node *&cr)`	Function header.
441-442	`  cf = NULL;` `  cr = NULL;`	The copy starts empty.
443	`  if (f == NULL)`	Empty source...
444	`    return;`	...gives an empty copy.
445	`  node *p = f;`	Walk from the front.
446	`  node *end = r->next;`	Stop after the rear.
447	`  do`	do...while (full ring).
449	`    dpushRear(ch, cf, cr, p->data);`	Insert the value at the rear of the copy, using the base insert.
450	`    p = p->next;`	Next value.
451	`  } while (p != end);`	Until past the rear.

8.7 Q8: disBalanced (balanced brackets)

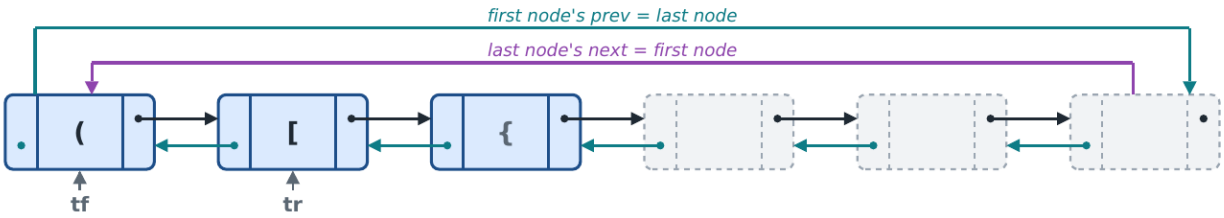
What it does: checks that every ({ [is closed by the matching) }] in the right order. The deque is used **like a stack**: opening brackets go in at the rear, and a closing bracket takes the **newest** one back from the rear.

Parameter	Passed how	Why the function needs it
s	by value (string)	The brackets to check. Uses dtmp with local tf, tr.
return value	int	1 = balanced, 0 = not.

Input ([{}]) : after the three opening brackets, the newest one { is at the rear



Closing } : dpopRear returns {, which matches. Then] matches [and) matches (. Empty at the end: Balanced



Line	Code (deque.cpp)	What this line does, and why
456	`int func::disBalanced(string s)`	Function header.
458	`  node *tf = NULL, *tr = NULL;`	Empty temporary deque.
459	`  int len = s.length();`	Number of characters.
460	`  for (int i = 0; i < len; i++)`	Look at each character.
462	`    char ch = s[i];`	The current character.
464	`    if (ch == '(' || ch == '{' || ch == '[')`	Opening bracket?
465	`      dpushRear(dtmp, tf, tr, ch);`	Insert it at the rear.
467	`    else if (ch == ')' || ch == '}' || ch == ']')`	Closing bracket?
469	`      if (tf == NULL)                // nothing to match with`	Nothing open to match it with...
470	`        return 0;`	...so not balanced.
471	`      char open = dpopRear(dtmp, tf, tr);`	Take the **newest** opening bracket from the rear.
472	`      if (ch == ')' && open == '(')`	Matching round brackets?
473	`        continue;`	Yes: go to the next character.
474	`      if (ch == '}' && open == '{')`	Matching curly brackets?
475	`        continue;`	Yes: next character.
476	`      if (ch == ']' && open == '[')`	Matching square brackets?
477	`        continue;`	Yes: next character.
478	`      return 0;                      // wrong pair`	No pair matched: wrong bracket, not balanced.

Line	Code (deque.cpp)	What this line does, and why
482	`if (tf == NULL)`	After all characters: is nothing left open?
483	`return 1;`	Balanced.
484	`return 0;`	Some opening bracket was never closed.

Q. Why remove from the rear and not the front?

The bracket to match is the most recent one, which is at the rear. Removing from the front would compare with the oldest bracket, which is wrong for `([)]`.

Q. What does `continue` do here?

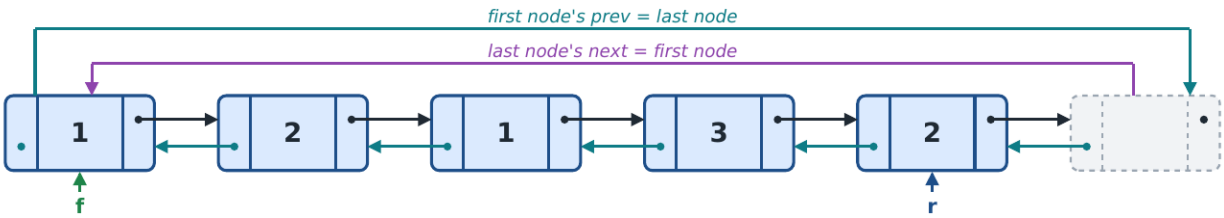
It skips the rest of the loop body and moves to the next character, because the pair matched.

8.8 Q9: dremoveDuplicates (remove duplicates)

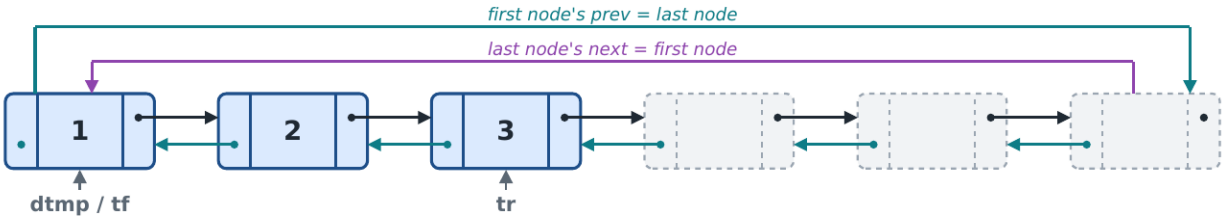
What it does: keeps only the **first** copy of each value, in the same order. It empties the deque into dtmp (skipping values already there), then moves everything back.

Parameter	Passed how	Why the function needs it
head	by value	Head of the deque's ring; needed to refill it with dpushRear.
f, r	by reference	The deque is emptied and refilled, so f and r change. The menu must see the new values.

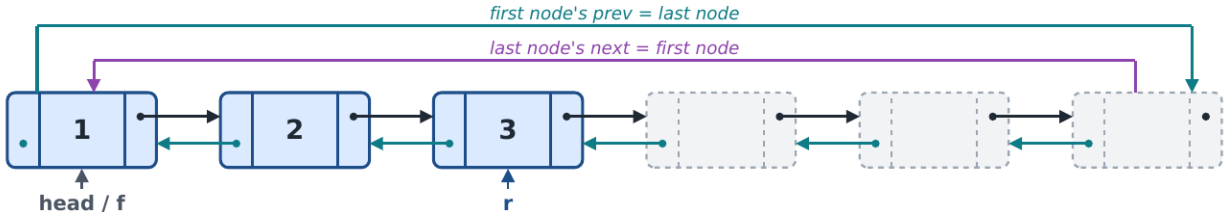
Before: 1 2 1 3 2



dtmp after the first loop: 1 2 3 (the second 1 and second 2 were skipped)



After moving back: 1 2 3



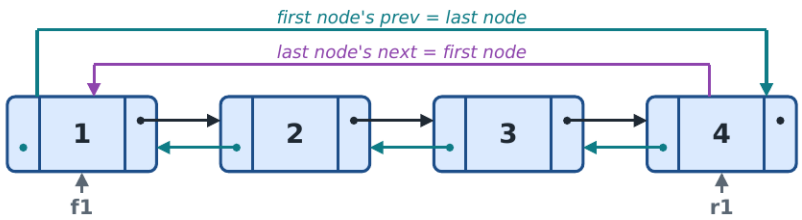
Line	Code (deque.cpp)	What this line does, and why
488	`void func::dremoveDuplicates(node *head, node *&f, node *&r)`	Function header.
490	`  node *tf = NULL, *tr = NULL;`	Empty temporary deque.
492	`  while (f != NULL)`	Until the deque is empty...
494	`    int x = dpopFront(head, f, r);`	...take the front value (oldest first).
495	`    if (dsearch(tf, tr, x) == 0)`	Not seen before?
496	`      dpushRear(dtmp, tf, tr, x);`	Keep it: insert at the rear of dtmp.
499	`  while (tf != NULL)`	Now move the kept values back...
501	`    int x = dpopFront(dtmp, tf, tr);`	...from the front of dtmp...
502	`    dpushRear(head, f, r, x);`	...to the rear of the deque. Same order as before, because front-to-rear is kept both times.

8.9 Q10: dintersection (values in both deques)

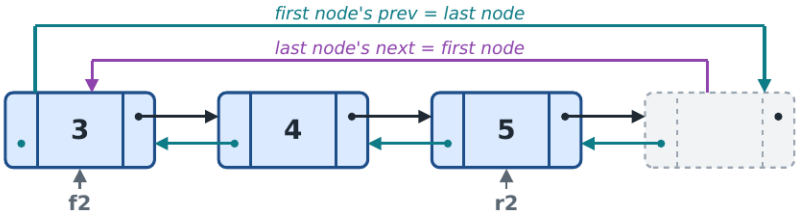
What it does: puts every value that appears in **both** deques into a third (result) deque, once each.

Parameter	Passed how	Why the function needs it
f1, r1	by value	The first deque: we walk through its values.
f2, r2	by value	The second deque: only searched.
rh	by value	Head of the result ring.
rf, rr	by reference	Front and rear of the result. Set by the inserts.

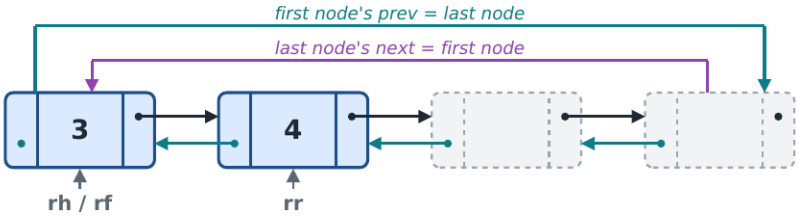
First deque: 1 2 3 4



Second deque: 3 4 5



Result: 3 4



Line	Code (deque.cpp)	What this line does, and why
507	`void func::dintersection(node *f1, node *r1, node *f2, node *r2, node *rh, node *&rf, node *&rr)`	Function header.
509-510	`  rf = NULL;` `  rr = NULL;`	The result starts empty.
513	`  if (f1 == NULL || f2 == NULL)`	If either deque is empty nothing can be common...
514	`    return;`	...so stop.
517	`  int n = dcount(f1, r1);`	Number of values in the first deque. The loop is counted because a ring has no NULL end.
518	`  node *p = f1;`	Start at the first deque's front.
519	`  for (int i = 0; i < n; i++)`	Once for each value.
522	`    if (dsearch(f2, r2, p->data) == 1 && dsearch(rf, rr, p->data) == 0)`	In the second deque **and** not already in the result? (&&: if the first test fails, the second is not run.)
523	`      dpushRear(rh, rf, rr, p->data);`	Then insert it at the rear of the result.
524	`    p = p->next;`	Next value.

Q. What does the second dsearch (on the result) prevent?

Adding the same value twice when it appears twice in the first deque.

Q. Who checks that the two deques exist and are different?

The menu (case 18), with four checks, before calling dintersection.

9. The menu: dequeMenu and menu

The functions above never ask the user anything. **dequeMenu** does the asking: it shows the choices, reads a number, checks the input and calls the right function on the **selected** deque.

9.1 The key idea: references to the selected deque

At the top of the loop (lines 539-541) three names are made: `h`, `f` and `r`. They are **references** to the array slots of the selected deque, not copies. So `dpushFront(h, f, r, x)` really moves `dfront[dcur]`. When the user selects another deque, `dcur` changes and on the next loop h, f and r refer to that deque instead.

Menu number	What it does	Function called
1 / 2	Insert at front / rear	dpushFront / dpushRear
3 / 4	Delete from front / rear	dpopFront / dpopRear
5 / 6	Display front to rear / rear to front	ddisplay / ddisplayBack
7 / 8 / 9	Count / Search / Reverse	dcount / dsearch / dreverse
10	Q1 split (adds 2 deques)	splitDeque
11	Q2 even / odd (adds 2 deques)	devenOdd
12	Q3 next greater	dnextGreater
13	Q5 sort	dsortList
14	Q6 palindrome	disPalindrome
15	Q7 copy (adds 1 deque)	copyDeque
16	Q8 brackets	disBalanced
17	Q9 remove duplicates	dremoveDuplicates
18	Q10 intersection (adds 1 deque)	dintersection
19 / 20 / 21	Create / Select / Show all deques	createList / (dcur) / ddisplay
0	Exit	

9.2 dequeMenu line by line

Line	Code (deque.cpp)	What this line does, and why
532	`void func::dequeMenu()`	Function header: the deque menu.
534	`int ch = 0, x = 0, n = 0;`	ch = menu choice, x = value to insert or search, n = deque number for 'select'.
535	`string s;`	Holds the word for palindrome and brackets.
536	`do`	Start of the menu loop: it always shows the menu at least once.

539-541	`node *&h = dhead[dcur];`
	`node *&f = dfront[dcur];`
	`node *&r = drear[dcur];`
		h, f, r are references to the selected deque's head, front and rear. Every function below works on them, so one set of functions serves all 100 deques.

Line	Code (deque.cpp)	What this line does, and why

543-566	`cout << "\n--- DEQUE " << dcur + 1 << " (of " << dnum << ") ---" << endl;`
	`cout << "1. Insert at front" << endl;`
	`cout << "2. Insert at rear" << endl;`
	`...`
	`cout << "0. Exit" << endl;`
	`cout << "Choice: ";`
		Print the title (which deque, how many exist) and the 21 choices.

567	`cin >> ch;`	Read the choice.
569	`switch (ch)`	Jump to the matching case.
571	`case 1:`	**1. Insert at front.**
572-573	`cout << "Value: ";` `cin >> x;`	Ask for the value.
574	`dpushFront(h, f, r, x);`	Insert at the front (its 4 checks handle empty and full).
575-576	`cout << "Deque " << dcur + 1 << ": ";` `ddisplay(f, r);`	Show the deque afterwards.
577	`break;`	Leave the switch; the loop shows the menu again.
579	`case 2:`	**2. Insert at rear.**
580-581	`cout << "Value: ";` `cin >> x;`	Ask for the value.
582	`dpushRear(h, f, r, x);`	Insert at the rear.
583-584	`cout << "Deque " << dcur + 1 << ": ";` `ddisplay(f, r);`	Show the deque.
585	`break;`	Back to the menu.
587	`case 3:`	**3. Delete from front.**
588-589	`if (f == NULL)` `cout << "Deque Underflow" << endl;`	If empty, print the underflow message here...
590-591	`else` `cout << "Deleted from front: " << dpopFront(h, f, r) << endl;`	...otherwise delete and print the value. (This avoids printing 'Deleted: -1'.)
592-593	`cout << "Deque " << dcur + 1 << ": ";` `ddisplay(f, r);`	Show the deque.
594	`break;`	Back to the menu.
596	`case 4:`	**4. Delete from rear.**
597-598	`if (f == NULL)` `cout << "Deque Underflow" << endl;`	Empty: underflow message...
599-600	`else` `cout << "Deleted from rear: " << dpopRear(h, f, r) << endl;`	...otherwise delete from the rear and print the value.
601-602	`cout << "Deque " << dcur + 1 << ": ";` `ddisplay(f, r);`	Show the deque.
603	`break;`	Back to the menu.
605	`case 5:`	**5. Display** front to rear.

Line	Code (deque.cpp)	What this line does, and why
606-607	`cout << "Deque " << dcur + 1 << " (front first): ";` `ddisplay(f, r);`	Print front to rear (follows next).
608	`break;`	Back to the menu.
610	`case 6:`	**6. Display backward** rear to front.
611-612	`cout << "Deque " << dcur + 1 << " (rear first): ";` `ddisplayBack(f, r);`	Print rear to front (follows prev). Only possible this simply because the list is doubly linked.
613	`break;`	Back to the menu.
615	`case 7:`	**7. Count.**
616	`cout << "Count: " << dcount(f, r) << endl;`	Print the number of values.
617	`break;`	Back to the menu.
619	`case 8:`	**8. Search.**
620-621	`cout << "Value to search: ";` `cin >> x;`	Ask for the value to look for.
622-623	`if (dsearch(f, r, x) == 1)` `cout << x << " found" << endl;`	dsearch returns 1: print found…
624-625	`else` `cout << x << " not found" << endl;`	…otherwise not found. (1 and 0 instead of bool.)
626	`break;`	Back to the menu.
628	`case 9:`	**9. Reverse.**

629-632	`if (f == NULL)`
	`cout << "Deque is empty, insert some values first" << endl;`
	`break;`
		Check: an empty deque has nothing to reverse.

634	`dreverse(f, r);  // f and r are references, so the new front and rear are kept`	Turn the links. f and r are references into the arrays, so the new front and rear are kept.
635-636	`cout << "Reversed deque " << dcur + 1 << ": ";` `ddisplay(f, r);`	Show the result: it starts from the new front, the old rear.
637	`break;`	Back to the menu.
639	`case 10:`	**10. Q1 split.** (Braces are needed because the case declares variables.)

641-644	`if (f == NULL)`
	`cout << "Deque is empty, insert some values first" << endl;`
	`break;`
		Check: an empty deque cannot be split.

646-649	`if (dnum + 2 > MAXS)`
	`cout << "No room for two more deques" << endl;`
	`break;`
		Check: there must be room for **two** new deques in the arrays.

652	`int a = dnum, b = dnum + 1;`	The two new deques go in the next two free slots.

Line	Code (deque.cpp)	What this line does, and why
653	`if (splitDeque(h, f, r, dhead[a], dfront[a], drear[a], dhead[b], dfront[b], drear[b]) == 0)`	Split; the halves are written straight into slots a and b.
655-656	`cout << "Deque has fewer than 2 nodes, it cannot be split" << endl;` `break;`	splitDeque returned 0: report it.
658	`dnum += 2;`	Two more deques now exist.

660-665	`cout << "Whole deque " << dcur + 1 << ": ";`
	`ddisplay(f, r);`
	`cout << "Deque " << a + 1 << " (first half): ";`
	`ddisplay(dfront[a], drear[a]);`
	`cout << "Deque " << b + 1 << " (second half): ";`
	`ddisplay(dfront[b], drear[b]);`
		Print the whole deque and both halves, with their numbers.

666	`break;`	Back to the menu.
669	`case 11:`	**11. Q2 even / odd.**

671-674	`if (f == NULL)`
	`cout << "Deque is empty, insert some values first" << endl;`
	`break;`
		Check: nothing to sort into even and odd.

676-679	`if (dnum + 2 > MAXS)`
	`cout << "No room for two more deques" << endl;`
	`break;`
		Check: room for two new deques.

682	`int e = dnum, o = dnum + 1;`	Slots for the even and the odd deque.

683-685	`createList(dhead[e]);`
	`dfront[e] = NULL;`
	`drear[e] = NULL;`
		Build the even deque's list and mark it empty.

686-688	`createList(dhead[o]);`
	`dfront[o] = NULL;`
	`drear[o] = NULL;`
		Build the odd deque's list and mark it empty.

689	`dnum += 2;`	Two more deques exist.
691	`devenOdd(f, r, dhead[e], dfront[e], drear[e], dhead[o], dfront[o], drear[o]);`	Fill them.

692-698	`cout << "Deque " << dcur + 1 << ": ";`
	`ddisplay(f, r);`
	`cout << "Even deque (deque " << e + 1 << "): ";`
	`ddisplay(dfront[e], drear[e]);`
	`cout << "Odd deque (deque " << o + 1 << "): ";`
	`ddisplay(dfront[o], drear[o]);`
	`cout << "Use 'Select a deque' to work on deque " << e + 1 << " or " << o + 1 << endl;`
		Print the original, the even and the odd deque, and how to select them.

699	`break;`	Back to the menu.

Line	Code (deque.cpp)	What this line does, and why
702	`    case 12:`	**12. Q3 next greater.**

703-706	`      if (f == NULL)`
	`        cout << "Deque is empty, insert some values first" << endl;`
	`        break;`
		Check: empty deque.

708-709	`      cout << "Deque " << dcur + 1 << ": ";` `      ddisplay(f, r);`	Show the deque first.
710	`      dnextGreater(f, r);`	Print every value's next greater element.
711	`      break;`	Back to the menu.
713	`    case 13:`	**13. Q5 sort.**

714-717	`      if (f == NULL)`
	`        cout << "Deque is empty, insert some values first" << endl;`
	`        break;`
		Check: empty deque.

719-720	`      cout << "Before: ";` `      ddisplay(f, r);`	Show before.
721	`      dsortList(f, r);`	Sort.
722-723	`      cout << "After: ";` `      ddisplay(f, r);`	Show after.
724	`      break;`	Back to the menu.
726	`    case 14:`	**14. Q6 palindrome.**
727-728	`      cout << "Enter string: ";` `      cin >> s;`	Read a word.
729-730	`      if ((int)s.length() > maxsize)` `        cout << "Deque Overflow: string is longer than " << maxsize << " characters" << endl;`	Check: the temporary deque has only 6 nodes, so a longer word would overflow.
731-732	`      else if (disPalindrome(s) == 1)` `        cout << "Palindrome" << endl;`	Otherwise test it…
733-734	`      else` `        cout << "Not Palindrome" << endl;`	…and print the answer.
735	`      break;`	Back to the menu.
737	`    case 15:`	**15. Q7 copy.**

739-742	`      if (f == NULL)`
	`        cout << "Deque is empty, insert some values first" << endl;`
	`        break;`
		Check: empty deque.

744-747	`      if (dnum + 1 > MAXS)`
	`        cout << "No room for another deque" << endl;`
	`        break;`
		Check: room for one new deque.

749	`      int c = dnum;              // the copy goes in a new deque`	The copy goes in the next free slot.

Line	Code (deque.cpp)	What this line does, and why

750-752	`createList(dhead[c]);`
	`dfront[c] = NULL;`
	`drear[c] = NULL;`
		Build its list and mark it empty.

753	`dnum++;`	One more deque exists.
755	`copyDeque(f, r, dhead[c], dfront[c], drear[c]);`	Copy the values.

756-759	`cout << "Original (deque " << dcur + 1 << "): ";`
	`ddisplay(f, r);`
	`cout << "Copy (deque " << c + 1 << "): ";`
	`ddisplay(dfront[c], drear[c]);`
		Print the original and the copy.

760	`break;`	Back to the menu.
763	`case 16:`	**16. Q8 balanced brackets.**
764-765	`cout << "Enter brackets: ";` `cin >> s;`	Read the brackets.
766-767	`if ((int)s.length() > maxsize)` `cout << "Deque Overflow: more than " << maxsize << " brackets" << endl;`	Check: too long for the 6-node temporary deque.
768-769	`else if (disBalanced(s) == 1)` `cout << "true (Balanced)" << endl;`	Balanced…
770-771	`else` `cout << "false (Not Balanced)" << endl;`	…or not. (Only text: no bool is used.)
772	`break;`	Back to the menu.
774	`case 17:`	**17. Q9 remove duplicates.**

775-778	`if (f == NULL)`
	`cout << "Deque is empty, insert some values first" << endl;`
	`break;`
		Check: empty deque.

780-781	`cout << "Before: ";` `ddisplay(f, r);`	Show before.
782	`dremoveDuplicates(h, f, r);`	Remove duplicates (f and r change, so the references h, f, r are passed).
783-784	`cout << "After: ";` `ddisplay(f, r);`	Show after.
785	`break;`	Back to the menu.
787	`case 18:`	**18. Q10 intersection** of two deques chosen by the user.
789	`int a = 0, b = 0;`	The two deque numbers.

Line	Code (deque.cpp)	What this line does, and why

790-793	`cout << "First deque (1 to " << dnum << "): ";`
	`cin >> a;`
	`cout << "Second deque (1 to " << dnum << "): ";`
	`cin >> b;`
		Ask for both numbers.

796-799	`if (a < 1 || a > dnum || b < 1 || b > dnum)`
	`cout << "Wrong deque number" << endl;`
	`break;`
		Check 1: both numbers must name deques that exist.

802-805	`if (a == b)`
	`cout << "Choose two different deques" << endl;`
	`break;`
		Check 2: the two deques must be different.

807-808	`a--;  // user numbers start at 1, the array at 0`
	`b--;`
		User numbers start at 1, the arrays at 0.

810-813	`if (dfront[a] == NULL)`
	`cout << "Deque " << a + 1 << " is empty, insert some values first" << endl;`
	`break;`
		Check 3 (first deque): must not be empty.

815-818	`if (dfront[b] == NULL)`
	`cout << "Deque " << b + 1 << " is empty, insert some values first" << endl;`
	`break;`
		Check 3 (second deque): must not be empty.

821-824	`if (dnum + 1 > MAXS)`
	`cout << "No room for another deque" << endl;`
	`break;`
		Check 4: room for the result deque.

827	`int res = dnum;  // the result goes in a new deque`	The result goes in the next free slot.

828-830	`createList(dhead[res]);`
	`dfront[res] = NULL;`
	`drear[res] = NULL;`
		Build its list and mark it empty.

831	`dnum++;`	One more deque exists.
833	`dintersection(dfront[a], drear[a], dfront[b], drear[b], dhead[res], dfront[res], drear[res]);`	Fill the result with the common values.

834-839	`cout << "Deque " << a + 1 << ": ";`
	`ddisplay(dfront[a], drear[a]);`
	`cout << "Deque " << b + 1 << ": ";`
	`ddisplay(dfront[b], drear[b]);`
	`cout << "Intersection (deque " << res + 1 << "): ";`
	`ddisplay(dfront[res], drear[res]);`
		Print both inputs and the result.

840	`break;`	Back to the menu.
843	`case 19:`	**19. Create a new deque.**

Line	Code (deque.cpp)	What this line does, and why

844-847	`if (dnum >= MAXS)`
	`  cout << "No room for another deque" << endl;`
	`  break;`
		Check: the arrays are not full.

849-851	`createList(dhead[dnum]);`
	`dfront[dnum] = NULL;`
	`drear[dnum] = NULL;`
		Build a new doubly circular list in the next slot, empty.

852	`dcur = dnum;  // the new deque becomes the selected one`	Select the new deque.
853	`dnum++;`	One more deque exists.
854	`cout << "Deque " << dcur + 1 << " created and selected" << endl;`	Tell the user its number.
855	`break;`	Back to the menu.
857	`case 20:`	**20. Select a deque.**
858-859	`cout << "Deque number (1 to " << dnum << "): ";` `cin >> n;`	Ask for the number.
860-861	`if (n < 1 || n > dnum)` `  cout << "Wrong deque number" << endl;`	Check: it must exist.
862	`else`	Valid number…
864	`  dcur = n - 1;`	…select it (array index = number - 1). On the next loop h, f, r refer to this deque.
865-866	`  cout << "Deque " << n << " selected: ";` `  ddisplay(dfront[dcur], drear[dcur]);`	Show it.
868	`break;`	Back to the menu.
870	`case 21:`	**21. Show all deques.**
871	`  for (int i = 0; i < dnum; i++)`	For every deque that exists…
873-874	`  cout << "Deque " << i + 1 << ": ";` `  ddisplay(dfront[i], drear[i]);`	…print its number and its values.
876	`  break;`	Back to the menu.
878-879	`case 0:` `  break;`	**0. Exit:** do nothing here; the loop condition below ends the loop.
881-882	`default:` `  cout << "Wrong choice" << endl;`	Any other number: tell the user.
884	`} while (ch > 0);`	Repeat while the choice is positive. 0 ends the menu and the program returns to main().

9.3 menu

Line	Code (deque.cpp)	What this line does, and why
887	`void func::menu()`	Start-up function called by main().
890	`  createList(dhead[0]);       // a doubly circular list of maxsize nodes`	Build deque 1's doubly circular list (array slot 0). Its front and rear are already NULL from the header, so it starts empty.
891	`  dnum = 1;`	One deque exists.
892	`  createList(dtmp);`	Build the temporary list used by palindrome, brackets and remove-duplicates.
894	`  dequeMenu();`	Open the deque menu loop.

Q. Why are h, f and r declared inside the loop and not before it?

References are bound once, when they are created. Declaring them inside the loop binds them again on every pass, so after 'Select a deque' changes dcur they refer to the newly selected deque.

Q. Why do some cases have braces { } and others not?

A case that declares variables (like `int a = dnum`) needs its own block, otherwise C++ complains that the jump to a later case skips the variable's initialisation.

Q. Why do cases 3 and 4 test f == NULL when dpopFront/dpopRear already check it?

Only to avoid printing 'Deleted from front: -1'. The menu prints the underflow message itself instead.

Q. What are the four checks before Q10?

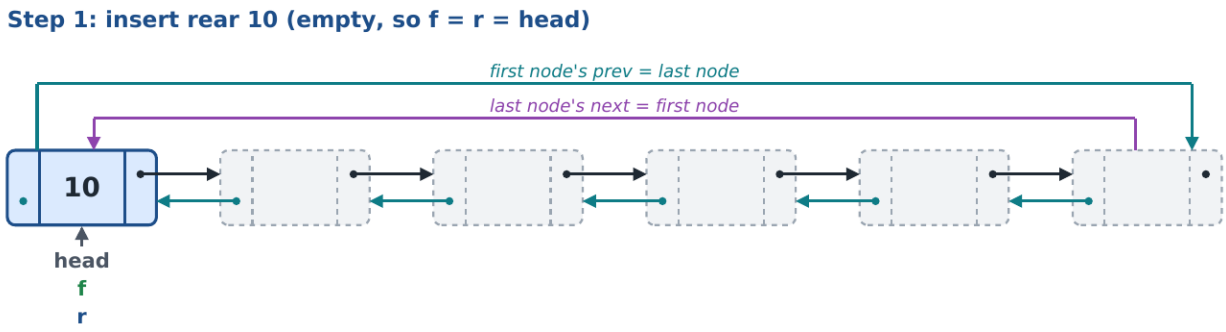
1. Both numbers exist. 2. They are different. 3. Neither deque is empty. 4. There is room for the result deque.

10. A complete dry run

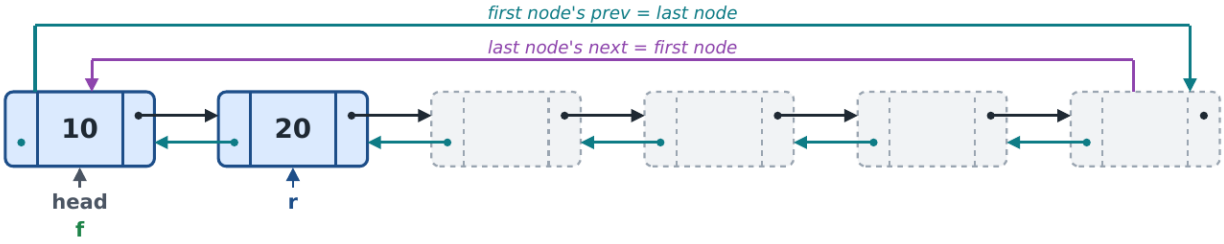
This is a real session (run on the compiled program). The 6 nodes are called N1 to N6; N1 is head and N6 is head->prev. Each step shows which check ran, where f and r are, and the output.

Step	Choice	Check that runs	f	r	Output (front to rear)
1	2 Insert rear 10	dpushRear Check 1 (empty)	N1	N1	10
2	2 Insert rear 20	dpushRear Check 4, r = r->next	N1	N2	10 20
3	1 Insert front 5	dpushFront Check 4, f on head: f = head->prev	N6	N2	5 10 20
4	1 Insert front 3	dpushFront Check 4, f = f->prev	N5	N2	3 5 10 20
5	2 Insert rear 30	dpushRear Check 4, r = r->next	N5	N3	3 5 10 20 30
6	2 Insert rear 40	dpushRear Check 4, r = r->next	N5	N4	3 5 10 20 30 40
7	1 Insert front 1	dpushFront Check 3: r->next == f	N5	N4	Circular Deque Overflow
8	6 Display backward	ddisplayBack: from r, follow prev	N5	N4	40 30 20 10 5 3
9	4 Delete rear	dpopRear Check 3, r = r->prev	N5	N3	Deleted from rear: 40
10	3 Delete front	dpopFront Check 3, f = f->next	N6	N3	Deleted from front: 3
11	3 Delete front	dpopFront Check 3, f on last node: f = head	N1	N3	Deleted from front: 5
12	4 Delete rear	dpopRear Check 3, r = r->prev	N1	N2	Deleted from rear: 30
13	4 Delete rear	dpopRear Check 3, r = r->prev	N1	N1	Deleted from rear: 20
14	3 Delete front	dpopFront Check 2 (one value)	NULL	NULL	Deleted from front: 10, then empty
15	3 Delete front	menu: f == NULL	NULL	NULL	Deque Underflow

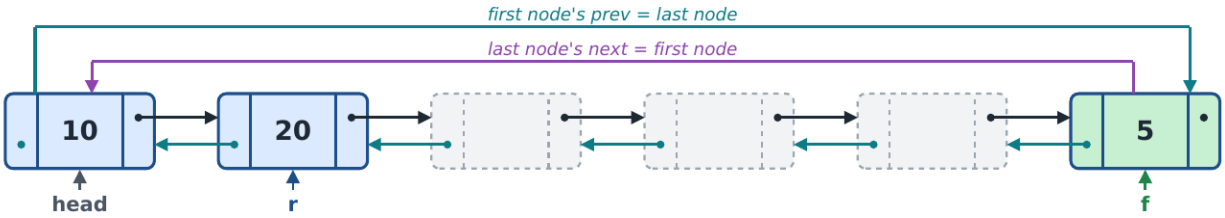
The same session in pictures



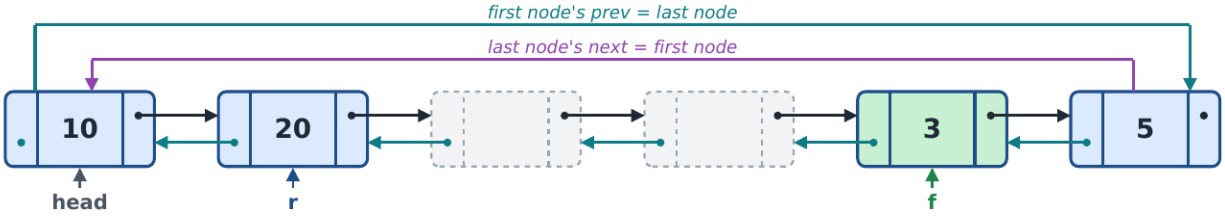
Step 2: insert rear 20



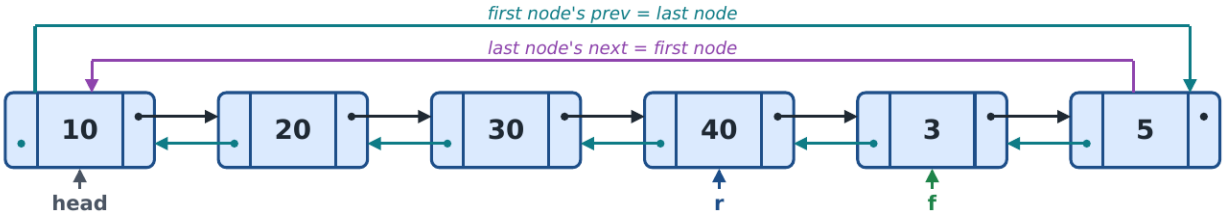
Step 3: insert front 5. f was on head, so f = head->prev (the last node)



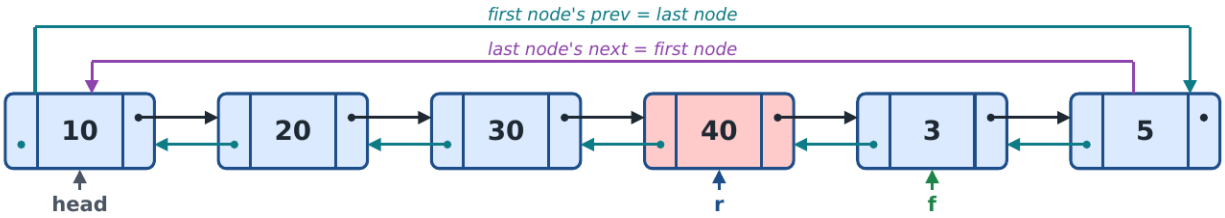
Step 4: insert front 3. f = f->prev, one node back



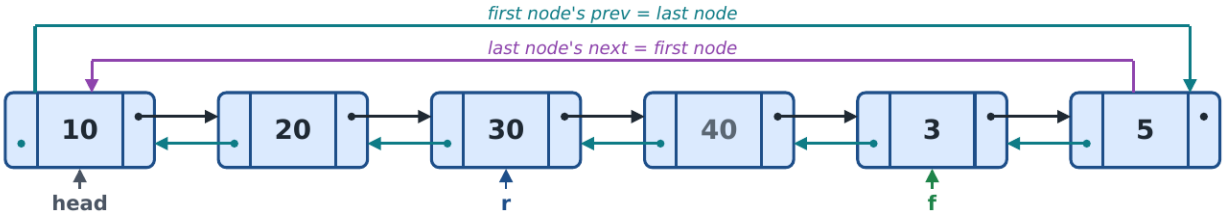
Steps 5-6: insert rear 30 and 40. Full: r->next is f



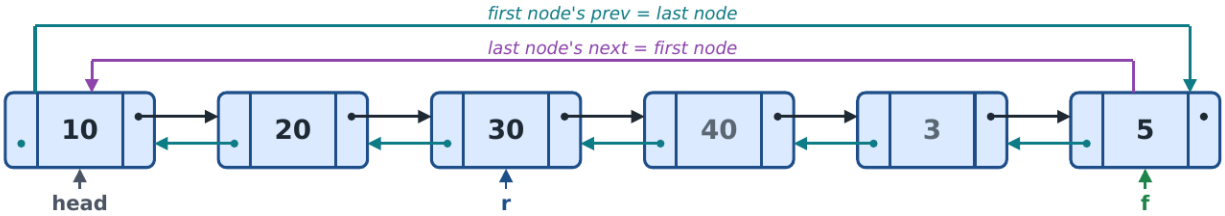
Step 7: insert front 1 gives Circular Deque Overflow (Check 3)



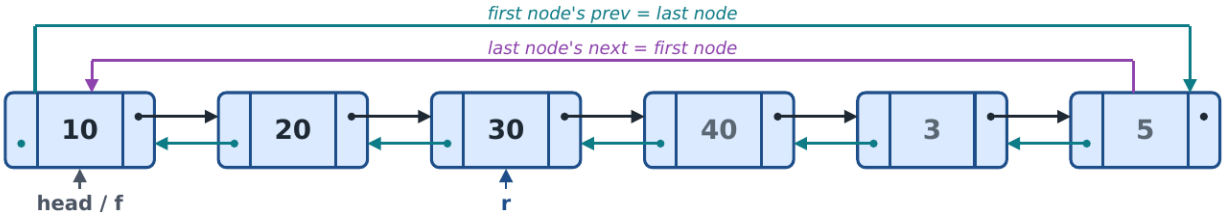
Step 9: delete rear (40). r = r->prev



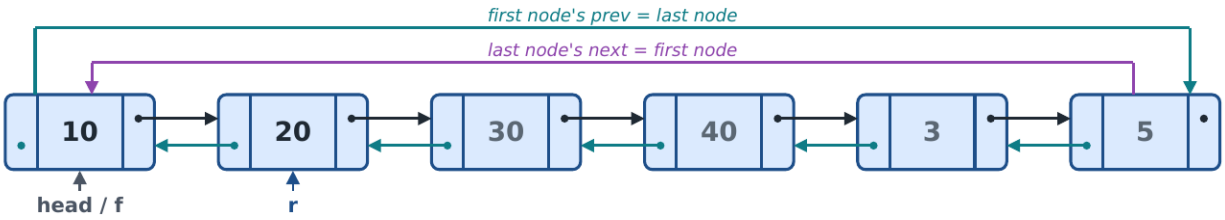
Step 10: delete front (3). f = f->next



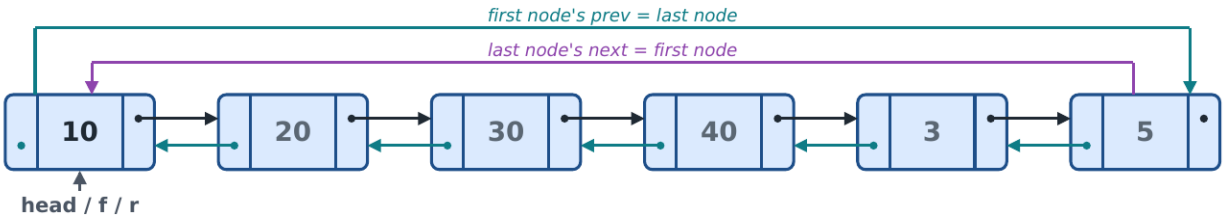
Step 11: delete front (5). f was on the last node, so f = head



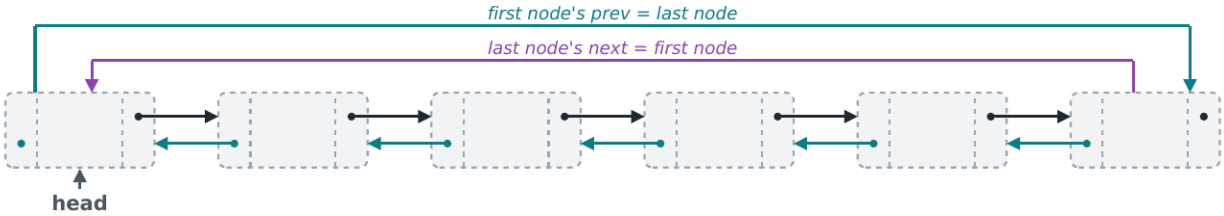
Step 12: delete rear (30)



Step 13: delete rear (20). Now f == r: one value



Steps 14-15: delete front (10) empties it (f = r = NULL). The next delete gives Deque Underflow



Grey numbers are old values in free nodes: they are still in memory but are not part of the deque.

11. Viva question bank

11.1 Concepts

Q. What is a deque?

A double-ended queue: values can be inserted and deleted at both the front and the rear.

Q. How is it different from a queue and a stack?

A queue inserts at the rear and deletes at the front. A stack inserts and deletes at the top. A deque allows all four: insert front, insert rear, delete front, delete rear.

Q. What are input-restricted and output-restricted deques?

Input-restricted: insert at one end only (rear), delete at both. Output-restricted: insert at both ends, delete at one end only (front). This program is unrestricted: all four operations.

Q. What is a doubly linked list?

A list where every node has two pointers: `next` to the node after it and `prev` to the node before it. You can walk it in both directions.

Q. What is a doubly circular linked list?

A doubly linked list whose ends are joined: the last node's `next` is the first node and the first node's `prev` is the last node. There is no NULL anywhere in the list.

Q. Why build a deque on a doubly circular list?

A deque moves its pointers both ways. With `prev`, moving back is one step (`f->prev`), so all four operations are O(1). The circular links let f and r wrap at both ends, so free space at either end is reused.

Q. What is the cost of the doubly linked list?

Each node stores one extra pointer, so it uses more memory, and every link has to be set in two directions (createList and splitDeque change both).

Q. What is the time complexity of the four operations?

O(1) each: every one does a fixed number of checks and moves a pointer by one step. Display, count and search are O(n).

Q. What is the capacity?

6 values per deque (maxsize = 6), because each list has 6 nodes. Up to 100 deques (MAXS).

Q. Is memory allocated on every insert?

No. createList makes all nodes once. Inserts and deletes only move f and r and write data.

Q. Can the deque work as a stack? As a queue?

Stack: dpushRear + dpopRear (or dpushFront + dpopFront). Queue: dpushRear + dpopFront.

11.2 Pointers and checks

Q. When is the deque empty?

When f == NULL (and r == NULL).

Q. When is it full?

Simple: f == head && r->next == head (front on the first node, rear on the last). Circular: r->next == f (front = rear + 1).

Q. When does it hold exactly one value?

When f == r (and they are not NULL).

Q. Which operations move a pointer backward, and with which link?

dpushFront (f = f->prev) and dpopRear (r = r->prev).

Q. Which move forward?

dpushRear (r = r->next) and dpopFront (f = f->next).

Q. How is the wrap handled moving forward?
r on the last node (r->next == head) goes to head; f on the last node (f->next == head) goes to head.

Q. How is the wrap handled moving backward?
If f (or r) is on head, it goes to head->prev, which is the last node.

Q. Why do both empties set f and r to NULL?
So the next insert's Check 1 sees an empty deque and starts again at the first node.

Q. What is returned when deleting from an empty deque?
-1, after printing Deque Underflow.

11.3 Questions about specific lines

Q. What does deque.h line 14 add?
The `prev` pointer. It is what makes the list doubly linked.

Q. What do lines 18-19 do?
Link the new node to the previous one in both directions.

Q. What do lines 22-23 do?
Close the list both ways: the last node's next is the first node and the first node's prev is the last node.

Q. What does line 39 do?
Check 1 of dpushFront: if the deque is empty, the value goes into the first node and f and r both point there.

Q. What does line 49 do?
Check 2 of dpushFront: front on the first node and rear on the last node means a full, not wrapped, deque: Deque Overflow.

Q. What does line 57 do?
Check 3 of dpushFront: the node after the rear is the front, so a wrapped deque is full: Circular Deque Overflow.

Q. What does line 66 do?
The front is on the first node, so it wraps back to the last node, which is head->prev.

Q. What does line 68 do?
Moves the front one node back using the prev link.

Q. What does line 104 test?
Whether the rear is on the last node; if so line 105 wraps it to the first node.

Q. Why is line 122 before the checks that move f?
To keep the value; once f moves, that node is no longer the front.

Q. What does line 165 test?
Whether the rear is on the first node; then it wraps back to head->prev.

Q. What does line 181 store?
The node just after the rear: the loop stops there. When the deque is full this is the front itself, which is why a do...while is used.

Q. What does line 199 store?
The node just before the front: the backward loop stops there.

Q. What does line 276 test?
a is the first free node after the old rear. If a is now the rear (the old front), the deque filled the ring, so there is no free node to join.

Q. What do lines 320 and 322 do?
They close the two halves in the backward direction, so each half is a complete doubly circular list.

Q. What do lines 539-541 do?
Make h, f, r references to the selected deque's array slots, so every function works on the selected deque.

11.4 C++ questions

Q. What is the difference between `node *f` and `node *&f`?
`node *f` is a copy of the pointer (by value): moving it inside the function does not affect the caller. `node *&f` is a reference to the caller's pointer: moving it changes the caller's pointer.

Q. What does `new node` do?
Allocates one node in memory and returns its address.

Q. What is NULL?
A pointer value meaning 'points to nothing'. Here it means an empty deque (f, r). Inside a circular list no link is NULL.

Q. Why do...while instead of while?
do...while runs the body before testing. For a full list the stop node equals the start node, so a while loop would run zero times.

Q. What does `%` do in `p->data % 2 == 0`?
Remainder after division. Remainder 0 when dividing by 2 means the number is even.

Q. What does `&&` do in Q10's condition?
Both tests must be true. If the first is false the second is not even run (short-circuit).

Q. What does `continue` do in disBalanced?
Skips the rest of the loop body and goes to the next character.

Q. What does `break` do in the switch and in dnextGreater?
In the switch it leaves the switch. In dnextGreater it leaves the inner for loop once the answer is found.

Q. Why `func::` before the function names in deque.cpp?
The functions belong to struct func; `func::` says which struct the body is for.

Q. Why `const int MAXS`?
Array sizes must be constants, and const stops it being changed.

Q. Why no `!` and no `bool`?
Lab rule. Conditions are written as comparisons (`f == NULL`, `== 0`) and results as 1 or 0.

11.5 What-if questions

Q. What if Check 1 were removed from dpushFront?
On an empty deque r is NULL, so `r->next` in Check 2 would crash the program.

Q. What if Check 4 came before Checks 2 and 3?
f would move before the full test; on a full deque it would land on the rear's node and overwrite a value.

Q. What if line 19 were missing?
No node would have a valid prev. The first `f = f->prev` would read garbage and crash or write into random memory.

Q. What if line 23 were missing?
head->prev would be unset. Inserting at the front when f is on head (or deleting from the rear when r is on head) would follow a garbage pointer.

Q. What if line 22 were missing?
The last node's next would be unset, so moving forward past the last node (display of a wrapped deque, rear wrap) would crash.

Q. What if splitDeque changed only the next links?
Each half's first node would still point back into the other half, so a backward move in one half would land in the other half.

Q. What if maxsize were 10?
Every list would have 10 nodes, so each deque would hold 10 values; overflow would come at the 11th insert. Nothing else changes.

Q. What if the user inserts a 7th value?

One of the two full checks catches it and prints an overflow message; nothing is changed.

12. One-page revision sheet

Operation	Check	Line	Condition	Action
dpushFront	1	39	f == NULL && r == NULL	f = r = head, store
	2	49	f == head && r->next == head	Deque Overflow
	3	57	r->next == f	Circular Deque Overflow
	4	65	f == head	f = head->prev, else f = f->prev; store
dpushRear	1	78	f == NULL && r == NULL	f = r = head, store
	2	88	f == head && r->next == head	Deque Overflow
	3	96	r->next == f	Circular Deque Overflow
	4	104	r->next == head	r = head, else r = r->next; store
dpopFront	1	116	f == NULL	Deque Underflow, return -1
	2	126	f == r	f = r = NULL, return x
	3	135	f->next == head	f = head, else f = f->next
dpopRear	1	146	f == NULL	Deque Underflow, return -1
	2	156	f == r	f = r = NULL, return x
	3	165	r == head	r = head->prev, else r = r->prev

Remember these five sentences

1. The deque is a doubly circular list of 6 nodes made once by createList(head); inserts and deletes only move f and r.
2. Empty: f == NULL. One value: f == r. Full: f == head && r->next == head, or r->next == f.
3. Forward moves (insert rear, delete front) use next; the wrap is 'on the last node, go to head'.
4. Backward moves (insert front, delete rear) use prev; the wrap is 'on head, go to head->prev'. No search, so all four are O(1).
5. The menu works on the selected deque through references h, f, r to dhead[dcur], dfront[dcur], drear[dcur].